

FantasyConsole Documentation

TurtleReader — ESP32-S3 Lua Game Console

INTRODUCTION

Introduction	3
--------------------	---

CARTRIDGE FORMAT

.turtlecart Format	6
--------------------------	---

SCENE FORMAT

Scene Format	10
Scene Text Labels	21
Object Identity & Visibility	26

LUA SCRIPTING API

Lua API Overview	30
ENTRY VM	32
Object Scripts	36
Animation	42
Physics	45
Input	50
Scene Script	53
State Store	58
Firmware Bridge	63

ASSETS

Sprite Spec	69
Asset Binary Formats	79

GUI

HUD Border	85
GUI Layers	93

TURTLESTUDIO

TurtleStudio Guide	106
--------------------------	-----

HARDWARE

Audio (M0 Bringup) 110

RCA Composite Driver (P4) 114



TurtleReader Fantasy Console

TurtleReader is a fantasy console targeting the ESP32-S3 microcontroller. Games are distributed as `.turtlecart` cartridges stored on a microSD card. Game logic runs on an embedded Lua 5.4 VM.

Hardware spec

Property	Value
Target chip	ESP32-S3 (S3N16R8)
Display	ILI9488 LCD via LovyanGFX
Resolution	164 × 124 logical pixels
Colors	32 palette indices (index 31 = always transparent)
Storage	microSD (FAT filesystem)
Input	8 GPIO buttons (directional + 4 action)
Scripting	Lua 5.4.6 (two independent VMs)

Coordinate system

The console uses **scene space** as the primary coordinate convention:

- Origin **(0, 0)** is at the **bottom-left** corner
- **Y increases upward** (math convention)

The raw framebuffer is raster (row 0 = top, Y down). Conversion:



```
xfb = sx
yfb = (H - 1) - sy    where H = 124
```

Use `spix(sx, sy, color)` for scene-space drawing. Use `pix(xfb, yfb, color)` only for raw framebuffer access.

Three packages

Package	Language	Purpose
<code>firmware/TurtleReader/</code>	C++ (Arduino)	Runs on ESP32-S3; reads cart, drives display, hosts Lua VMs
<code>firmware/libraries/lua54/</code>	C	Vendored Lua 5.4.6, installed as Arduino library
<code>tools/turtlestudio/</code>	Python	Authoring tool – CLI + Dear PyGui GUI for building projects

Quick start

1. Install the `Lua54` Arduino library from `firmware/libraries/lua54/`.
2. Flash `firmware/TurtleReader/TurtleReader.ino` to an ESP32-S3. Enable OPI PSRAM.
3. Build a project with TurtleStudio and copy the `build/` folder to the microSD root.
4. Insert the microSD and power on.

Serial monitor runs at 115200 baud and outputs `print()` calls from Lua.

Next steps

- Cartridge format – how `.turtlecart` files are structured
- Lua API overview – ENTRY VM and actor script VMs

- Asset binary formats – `.tsp`, `.tbg`, `.tts`, `.tfn`
- TurtleStudio guide – authoring tool reference

.turtlecart Cartridge Format (v0)

A `.turtlecart` file is a plain-text container with an ordered set of sections. It's designed for simplicity: readable on any text editor, easy to parse on the ESP32.

File structure

```
TURTLECARD:0
ENTRY:scripts/global.lua
INITIAL_SCENE:intro
BUNDLE_FILE:studio/project_bundle.json
PALETTE:
#1a1a2e
#16213e
#0f3460
#e94560
...
---FILE:scripts/global.lua---
print("hello from entry")
cls(1)
flip()
---END---
```

Rules (v0)

#	Rule
1	First line must be exactly <code>TURTLECARD:0</code>
2	<code>ENTRY:<path></code> – Lua file to execute at boot (must be embedded below)
3	<code>INITIAL_SCENE:<id></code> – optional; scene to load after ENTRY runs (default: <code>intro</code>). <code>main</code> is a reserved id – do not use as a scene id.

#	Rule
4	<code>BUNDLE_FILE:<path></code> – optional; sidecar JSON bundle on the SD card. Do not embed the bundle in the cart – saves RAM on ESP32.
5	<code>PALETTE:</code> block – optional; one <code>#RRGGBB</code> or <code>#RGB</code> color per line. Up to 32 entries are applied to indices <code>0..31</code> . Fewer entries leave remaining indices as <code>#000000</code> .
6	Embedded files: content between <code>---FILE:<path>---</code> and <code>---END---</code>
7	The file named in <code>ENTRY</code> must exist as an embedded block

Recommended order: `TURTLE CART:` → `ENTRY:` → `INITIAL_SCENE:` →
`BUNDLE_FILE:` → `PALETTE:` → `---FILE:<ENTRY>---`

Palette

- `#RRGGBB` hex (upper or lowercase)
- `#RGB` shorthand is also accepted (each nibble is doubled)
- Blank lines and invalid lines are skipped
- Only the first 32 valid entries are applied at runtime; the rest are ignored
- Index 31 is always transparent – never paint with it

Default palette (when `PALETTE:` block is absent): a Genesis-style default is loaded by the firmware.

SD package layout

TurtleStudio exports a `build/` folder that gets copied to the microSD root:

```
build/
├─ main.turtlecart      # cart with embedded ENTRY Lua
```

```
|— studio/
|   └─ project_bundle.json  # scene/object/sprite manifest
|— backgrounds/*.tbg       # baked background images
|— sprites/*.tsp           # baked sprite sheets
|— tiles/*.tts             # baked tilesets
|— fonts/*.tfn             # baked fonts
|— objects/*.json          # object definitions
|— scripts/*.lua           # game logic scripts
|— COPIAR_A_SD.txt         # instructions (copy all of build/ to SD root)
```

Boot sequence

1. Mount microSD, load `main.turtlecart` (fallback: `demo.turtlecart`)
2. Load sidecar `studio/project_bundle.json`
3. Apply `PALETTE:` block to framebuffer (indices 0-31)
4. Execute embedded ENTRY Lua block (once, then destroy that VM)
5. `turtle_scene_begin_runtime` draws the `INITIAL_SCENE` (background, tiles, sprites)
6. `flip()` – display the first frame

NOTE

A `cls()` or drawing done in ENTRY won't persist: the scene runtime repaints the framebuffer at step 5. ENTRY is for initialization, not rendering persistent content.

Scene coordinate system

Scene space: 164 × 124, origin (0, 0) at bottom-left, Y positive upward. See [Intro](#) for the framebuffer conversion formula.

Out of scope in v0

- Compression, checksums, digital signatures

- Sprites embedded in the cart binary (they go as sidecar `.tsp` files)
- Binary container format with a file table (planned for v1+)

Scene Format (v0 / v1 / v2)

A scene defines the visual space where a game takes place: the coordinate system, camera, background layers, tile layers, and placed objects. Three incremental spec versions exist – each is **additive and backward-compatible** with `TURTELCART:0`; no field in any version is required.

Coordinate system

The console uses **scene space** as the primary game coordinate convention:

Property	Value
Viewport	164 × 124 logical pixels
Origin (0, 0)	Bottom-left corner
X axis	Positive → right
Y axis	Positive → up (math convention)
Valid X	<code>0..163</code>
Valid Y	<code>0..123</code>

The raw framebuffer is raster (Y down, row 0 = top). Conversion from scene to framebuffer coordinates:

```
xfb = sx
yfb = (H - 1) - sy    where H = 124
```

Use `spix(sx, sy, color)` in Lua for scene-space drawing. Use `pix(xfb, yfb, color)` only for raw framebuffer work.

Transparency: palette index 31 is always transparent. The runtime never copies pixels at index 31 when blitting sprites or backgrounds.

World size

The world can be larger than the viewport. `world_steps_x` and `world_steps_y` (integers 1-8) multiply the viewport dimensions:

- `world_steps_x: 8` → world width = 1312 px (8 screens wide)
- Default: `1` on both axes → world = viewport (no scrolling)

Resident window (how the firmware handles large worlds)

The firmware does not keep a pixel buffer for the entire world in RAM. Instead it maintains a **3×3-step resident window** (492×372 px, 183 024 bytes) centered on the camera. When the camera approaches the window edge, the firmware recenters and re-bakes the window – not every frame, approximately every ~164 px of scroll at normal game speed.

The tile grid (`uint8_t` per 16px cell) is kept fully resident for the entire world (≤20 KB for 4 layers at 8×8 steps). Collision and `posx()` / `posy()` always operate against the full tile grid – the pixel window does not affect them.

NOTE

Worlds of 1-2 steps fit entirely inside the 3-step window, so the initial bake is the only one needed for the life of the scene. The streaming mechanism is invisible for small scenes.

Camera

Configured in `scenes/<id>.json`, field `camera`:

Field	Default	Description
<code>mode</code>	<code>"follow"</code>	<code>"follow"</code> or <code>"fixed"</code>
<code>target</code>	<code>" "</code>	Id of object to follow. Falls back to <code>character</code> , then <code>player</code> , then the first object in <code>objects[]</code> .
<code>x</code> , <code>y</code>	—	Bottom-left corner of the viewport in scene space. Starting position for <code>follow</code> , fixed position for <code>fixed</code> .
<code>margin_x</code> , <code>margin_y</code>	—	Pixels from the viewport edge before the camera moves (<code>follow</code> only).

Background layers (`background_layers`)

A single array of 4 **layers** controls all background imagery. **Layer 1** (index `0`) is special: its background image is baked together with tiles into the static world buffer once per scene. **Layers 2-4** (indices `1..3`) each live in their own buffer and are repainted every frame.

```
"background_layers": [  
  { "enabled": true, "color_index": 1, "opacity": 255, "background":  
    "sky_main" },  
  { "enabled": true, "color_index": 1, "opacity": 255, "background":  
    "clouds_far",  
    "parallax_x": 0.2, "repeat_x": true },  
  { "enabled": true, "color_index": 1, "opacity": 255, "background":  
    "hills_mid",  
    "parallax_x": 0.5, "repeat_x": true },  
  { "enabled": false, "color_index": 1, "opacity": 255 }  
]
```

Field	Description
<code>enabled</code>	Whether this layer is drawn
<code>background</code>	Id of a background asset (<code>backgrounds/<id>.tbg</code>). Empty = color-fill only.
<code>color_index</code>	Palette index for the <code>cls()</code> fill-color (used by TurtleStudio for preview; not mixed with the image in v0)
<code>opacity</code>	<code>0..255</code> . In v0, only <code>255</code> (opaque) and <code>0</code> (invisible) are distinct at runtime. v1 adds 4-level Bayer dither; v2 adds real alpha blend on capable hardware.
<code>parallax_x</code>	Horizontal scroll factor for layers 2-4 (<code>0.0..2.0</code> ; <code>1.0</code> = same speed as camera). No effect on layer 1 – use <code>parallax_bands</code> instead.
<code>fixed</code>	<code>false</code> by default. If <code>true</code> , layer ignores camera entirely (offset always 0).
<code>repeat_x</code>	If <code>true</code> , the image wraps horizontally instead of clipping at its edge. Needed for slow-moving layers (<code>parallax_x < 1</code>) that would otherwise run out of source pixels.

Layer anchoring: all layers are anchored at the world's bottom-left (`scene_y = 0`). Scene rows above the image's pixel height are not painted by that layer.

Rendering cost (important)

- **Layer 1** – baked once into the static world buffer alongside tiles. Zero per-frame cost.
- **Layers 2-4** – each requires its own RAM buffer (`pixel_w × pixel_h` bytes in PSRAM) and a full repaint pass every frame.
- **Tile baking trade-off:** when any layer 2-4 is enabled, the firmware skips tile pre-baking and redraws tiles live every frame. The same applies when `parallax_bands` is present. This is an intentional

architectural choice – decorative tiles in disabled-collision layers work correctly, but come with this CPU cost.

Parallax bands (`parallax_bands`) – layer 1 only

`parallax_bands` divides the layer 1 background into horizontal Y-ranges, each with its own independent scroll factor. Classic platformer parallax effect (clouds barely move, hills at medium speed, foreground at full speed).

```
"parallax_bands": [
  { "y0": 88, "y1": 123, "parallax_x": 1.0 },
  { "y0": 44, "y1": 87, "parallax_x": 0.5, "repeat_x": true },
  { "y0": 0, "y1": 43, "parallax_x": 0.15, "repeat_x": true }
]
```

Field	Default	Description
<code>y0</code> , <code>y1</code>	–	Scene Y range (inclusive, Y up). Clamped to <code>[0, world_h-1]</code> ; swapped if <code>y0 > y1</code> .
<code>parallax_x</code>	–	Horizontal scroll factor, <code>0.0..2.0</code> . <code>1.0</code> = camera speed.
<code>fixed</code>	<code>false</code>	Ignores camera; horizontal offset always 0.
<code>repeat_x</code>	<code>false</code>	Wraps source image column with <code>modulo image_width</code> .

Limit: 8 bands per scene. Rows not in any active band use

`parallax_x=1.0`, `fixed=false`, `repeat_x=false`.

 CAUTION

As soon as any `parallax_bands` entry is defined, the firmware disables tile baking for the whole scene (same CPU trade-off as enabling layers 2-4).

Tile layers (`tile_layers`)

Up to 4 tile layers per scene. Each layer:

Field	Description
<code>enabled</code>	Whether the layer is drawn and its collision is evaluated
<code>tileset</code>	Stem of a tileset file (<code>tiles/<tileset>.tts</code>)
<code>cells</code>	Grid of tile indices. Empty cell = index 31 (transparent, no collision)

Collision layer: only one tile layer blocks actors. Set with `collision_tile_layer` (integer `0-3`, default `0`) at the scene level. The other three layers are purely visual – their tiles' `collision` metadata is ignored by the firmware.

Per-tile collision shapes

Defined in `tiles/<id>.json`, field `collision` per tile entry:

Value	Meaning
omitted or <code>"solid"</code>	Full cell, solid (default)
<code>"none"</code>	Decoration – no blocking
<code>{ "mode": "aabb", "x0", "y0", "x1", "y1" }</code>	Axis-aligned box in tile-local space (origin bottom-left, Y up)

Optional `oneway` (bool) + `oneway_direction` (`"up"` | `"down"` | `"left"` | `"right"`) for one-way platforms. `triangle` and `hexagon` shapes are accepted in JSON but approximated as AABB by the firmware.

Tile size is a multiple of 8 px (default 8 px, same as Game Boy/Game Gear). TurtleStudio draws a tile grid overlay in the scene editor.

Scene manifest fields (TurtleStudio)

Each scene in `turtlestudio.json` and `scenes/<id>.json`:

Field	Description
<code>id</code>	Unique scene identifier. <code>main</code> is reserved – do not use as a scene id.
<code>script</code>	Stem of <code>scripts/<stem>.lua</code> – the scene VM script (ticks before actors, no actor attached).
<code>palette</code>	Relative path to a palette file (<code>palettes/<name>.txt</code>).
<code>background_index</code>	Palette index for the <code>cls()</code> background fill (preview and runtime).
<code>world_steps_x</code> / <code>world_steps_y</code>	World size multiplier, <code>1..8</code> .
<code>camera</code>	Camera config object (see Camera above).
<code>background_layers</code>	Array of 4 layer configs (see above).
<code>parallax_bands</code>	Array of layer-1 parallax bands (see above).
<code>tile_layers</code>	Array of up to 4 tile layer configs.

Field	Description
<code>collision_tile_layer</code>	Index (0-3) of the tile layer that blocks actors. Default <code>0</code> .
<code>objects</code>	Array of placed object instances. Each has <code>object</code> (catalog id), <code>id</code> (instance id), <code>tags</code> (string array), <code>x</code> , <code>y</code> (scene-space).
<code>default_anim_fps</code>	Default animation frame rate for actors in this scene.

The `active_scene` field in `turtlestudio.json` marks the scene shown in TurtleStudio for editing.

v1 extensions – extended parallax

(*spec*: `spec/scene-v1.md`) All fields are optional; absent fields reproduce v0 behavior exactly.

Vertical parallax on layer-1 bands

Each `parallax_bands` entry gains:

Field	Default	Description
<code>parallax_y</code>	<code>1.0</code>	Vertical scroll factor (<code>0.0..2.0</code>). <code>1.0</code> = follows camera Y at full speed.
<code>repeat_y</code>	<code>false</code>	Wraps source image row with <code>modulo image_height</code> . Needed when <code>parallax_y < 1</code> .

`fixed: true` now suppresses both X and Y offsets (a fixed band is fixed on both axes).

Per-band parallax on layers 2-4

Each layer 2-4 entry can declare its own `parallax_bands` array (same schema as the scene-level one, including the new `parallax_y` / `repeat_y` fields). If present and non-empty, it overrides the layer's uniform `parallax_x` / `fixed` / `repeat_x` .

```
{ "enabled": true, "background": "hills_mid",
  "parallax_bands": [
    { "y0": 100, "y1": 123, "parallax_x": 0.6 },
    { "y0": 0,   "y1": 99,  "parallax_x": 0.4, "repeat_x": true }
  ]
}
```

Layer 1 (index 0) ignores an inline `parallax_bands` – it always uses the scene-level one.

Vertical scroll / offset for layers 2-4

Field	Default	Description
<code>parallax_y</code>	<code>1.0</code>	Vertical scroll factor for this layer
<code>repeat_y</code>	<code>false</code>	Wraps vertical sampling
<code>offset_y</code>	<code>0</code>	Fixed vertical offset in scene pixels, applied before <code>parallax_y</code> . Replaces the hard-coded bottom-left anchor of <code>v0</code> .

Layers in non-scrolling scenes

In `v0` , layers 2-4 were only rendered in scenes with scroll (`world_steps > 1`). `v1` removes this restriction – an `enabled: true` layer renders in any scene. In a non-scrolling scene, offsets are computed with `cam_x = cam_y = 0` , producing a static overlay.

Opacity (4-level Bayer dither)

v1 resolves `opacity` at runtime instead of ignoring non-255 values. The firmware maps `opacity` to 5 steps (`0`, `63`, `127`, `191`, `255`) and uses a Bayer 4×4 threshold pattern to paint only a fraction of pixels – classic indexed-palette pseudo-transparency, no framebuffer read required.

v2 extensions — `GFX_TIER`

(spec: `spec/scene-v2.md`) `GFX_TIER` is a firmware compile-time constant (`#define`), not a cartridge field.

Tier	Hardware target	<code>opacity</code> rendering
<code>0</code>	ESP32-S3 (current)	Bayer 4×4 dither (v1)
<code>1</code>	ESP32-P4 (planned)	Real alpha blend via PPA hardware

Compatibility rule

A cartridge authored for tier 1 must remain **fully playable** on tier 0 – with lower fidelity (dither instead of real blend) but never with broken collision, missing assets, or a scene that fails to load. Tier only affects visual fidelity and performance headroom, never game logic.

`GFX_TIER` in Lua

The ENTRY VM exposes `GFX_TIER` as a read-only global integer (alongside `W`, `H`, `COLORS`):

```
if GFX_TIER >= 1 then
  -- enable a decorative effect that relies on real alpha blend
end
```

Use it only for optional visual enhancements – never to gate mechanics or content.

`min_gfx_tier` **field**

A `background_layers` entry can declare `min_gfx_tier` (int, default `0`). If the running firmware's `GFX_TIER` is below this value, the layer is treated as `enabled: false` – a clean skip, not a failed load. No current feature requires `min_gfx_tier > 0`; the field is defined now so the cart format doesn't need to change when one appears.

Quick-reference: new fields across all versions

Version	Scope	Field	Type	Default
v1	<code>parallax_bands[]</code>	<code>parallax_y</code>	float	<code>1.0</code>
v1	<code>parallax_bands[]</code>	<code>repeat_y</code>	bool	<code>false</code>
v1	<code>background_layers[]</code> (2-4)	<code>parallax_bands</code>	array	<code>[]</code>
v1	<code>background_layers[]</code> (2-4)	<code>parallax_y</code>	float	<code>1.0</code>
v1	<code>background_layers[]</code> (2-4)	<code>repeat_y</code>	bool	<code>false</code>
v1	<code>background_layers[]</code> (2-4)	<code>offset_y</code>	int	<code>0</code>
v1	<code>background_layers[]</code> (any)	<code>opacity</code>	u8	<code>255</code>
v2	ENTRY Lua global	<code>GFX_TIER</code>	int	<code>0</code>
v2	<code>background_layers[]</code> (any)	<code>min_gfx_tier</code>	int	<code>0</code>
v2	TurtleStudio manifest	<code>recommended_gfx_tier</code>	int	<code>0</code>

Scene Text Labels (v0)

Scene text labels are static text strings baked directly into the scene JSON and drawn automatically by the firmware – no Lua code required. Use them for level names, tutorial hints, fixed UI text, or any string that doesn't need to change at runtime.

Bundle format

`text_labels` is an array inside the scene object. Maximum 16 labels per scene.

```
{
  "id": "level_1",
  "text_labels": [
    {
      "id": "title",
      "text": "World 1-1",
      "x": 10,
      "y": 110,
      "font": "font8",
      "color_index": -1
    },
    {
      "id": "hint",
      "text": "Jump over gaps!",
      "x": 10,
      "y": 4,
      "font": "font8",
      "color_index": 5,
      "blink_ms": 800
    }
  ]
}
```

Label fields

Field	Type	Default	Description
<code>id</code>	string	required	Unique identifier within this scene. Max 32 chars.
<code>text</code>	string	<code>""</code>	The string to draw. Max 63 chars.
<code>x</code>	int	<code>0</code>	Scene X coordinate of the left edge of the first glyph.
<code>y</code>	int	<code>0</code>	Scene Y coordinate of the bottom of the first glyph (scene space: origin bottom-left, Y upward).
<code>font</code>	string	required	Stem of a <code>.tfn</code> font asset in the bundle. Label is silently skipped if the font is missing.
<code>color_index</code>	int	<code>-1</code>	<code>-1</code> = use each glyph's own palette colors. <code>0..30</code> = flat tint applied to every non-transparent pixel in every glyph. Index 31 is transparent and has no effect as a tint.
<code>blink_ms</code>	int	<code>0</code>	Blink period in milliseconds. <code>0</code> = no blink (always visible). See Blinking labels below.

Supported charset: space, `A-Z`, `a-z`, `0-9`, and `. , ! ? : ; ' -`. Characters outside this set are silently skipped (the character occupies no width).

Coordinate system

`(x, y)` is in **scene space**: origin at the bottom-left of the effective playfield, Y increasing upward. The firmware converts to framebuffer coordinates internally.

```
scene (x=10, y=110) with hud_border.top=16, kSceneH=124:
  framebuffer_y = (kSceneH - 1) - y = 123 - 110 = 13
  framebuffer coords: (10 + L, 13 + T)
```

Rendering

Labels are part of the scene's **static layer** – drawn by `paint_scene_static_layers` alongside the background and tile layer.

Fixed camera (no scroll): Non-blinking labels are baked into `snapshot_static` during scene start. They cost zero per-frame repaint – the dirty-rect system preserves them automatically. Blinking labels are excluded from the snapshot and updated each frame using the dirty-rect system.

Scrolling camera: All labels are repainted every frame as part of the full scene repaint. Blink timing works the same way.

Labels are drawn **before** actors, so actors always composite on top of text.

Blinking labels

Setting `blink_ms` to a non-zero value makes the label blink symmetrically: the label is **visible** for `blink_ms / 2` ms, then **invisible** for `blink_ms / 2` ms, starting visible when the scene loads.

```
{ "id": "press_start", "text": "Press Start", "blink_ms": 1000, ... }
```

- The blink timer is reset on scene load.
- Blink state is independent per label – two labels with the same `blink_ms` may drift out of phase if the scene was entered at different times.
- TurtleStudio caps `blink_ms` at **60000** (1 minute) on save.
- `blink_ms: 0` means always visible (default, no blink behavior, no timer overhead).

Blinking labels are **never** included in `snapshot_static`. On fixed-camera scenes they use the dirty-rect mechanism: only the cells they occupy are dirtied and restored each frame they change state.

TIP

Blinking text is commonly used for "Press Start" title screens and countdown prompts. Keep blink periods at 500ms-1000ms for readability – faster than 200ms is generally unreadable on the small display.

Interaction with actors

Labels are part of the background layer – they do not interact with the physics or collision system. An actor can walk in front of a label (the actor sprite composites on top). There is no API to hide or update a label at runtime in v0; use [GUI layers](#) for dynamic text.

TurtleStudio

- The scene editor shows text labels as overlays on the canvas at their scene coordinates.
- Font, color, and blink period are editable inline.
- Preview mode blinks labels at the configured period.

Out of scope in v0

- Runtime mutation (`set_text`, `hide_label`) – use GUI layer `text_labels` for runtime-updatable text
- Scroll/marquee animation
- Per-character color

Object Identity & Visibility (v0)

Objects placed in a scene can have an **instance identity** (`id`, `tags`) for runtime lookup, and a **visibility flag** that controls whether the object draws.

Instance identity

Fields

Field	Type	Default	Description
<code>object</code>	string	–	Catalog reference – the stem in <code>objects/<stem>.json</code> . Tells the firmware which sprite, script, and collision shape to use.
<code>id</code>	string	<code>""</code>	Instance identifier. Unique within the scene. Used by <code>find_by_id</code> . Max 32 chars.
<code>tags</code>	string[]	<code>[]</code>	Arbitrary labels for group lookup via <code>find_by_tag</code> . Up to 6 tags per instance, each max 20 chars.

`object` and `id` are separate concerns: the same `object` catalog entry (e.g. `"coin"`) can be placed many times in a scene, each with a distinct `id` and different `tags`.

```
{
  "object": "coin",
  "id": "coin_01",
  "tags": ["collectible", "level1"],
  "x": 80, "y": 40
}
```

Legacy compatibility: if a scene object dict has no `"object"` key, the firmware uses `"id"` as the catalog reference (pre-v0 behavior where id doubled as the type). New projects should always include `"object"`.

Runtime Lua API

All handle-based functions are available from both ENTRY and actor VMs.

Function	Description
<code>find_by_id(id)</code> → handle nil	Find the actor instance with the given <code>id</code> . Returns <code>nil</code> if not found or already destroyed.
<code>find_by_tag(tag)</code> → table	Find all visible actor instances that have <code>tag</code> in their <code>tags</code> array. Returns an empty table if none.
<code>obj_posx(handle)</code> → number	X position of the actor in scene coordinates.
<code>obj_posy(handle)</code> → number	Y position of the actor in scene coordinates.
<code>obj_id(handle)</code> → string	The instance <code>id</code> string of the handle.
<code>obj_has_tag(handle, tag)</code> → bool	Test whether the handle has a specific tag.

Handles are **opaque** – do not store them across scene changes. A handle is invalidated when the scene ends or the actor is destroyed; using a stale handle is a silent no-op (returns `nil` / `false`).

```
-- Example: deactivate all "coin" actors on collection
local coins = find_by_tag("collectible")
for _, h in ipairs(coins) do
  if obj_posx(h) > 60 and obj_posx(h) < 90 then
    -- actor is near the player; do something
  end
end
```

```
end
```

NOTE

In v0 all handle data is **read-only** – you can query position, id, and tags, but cannot move, destroy, or modify actors via handles. Use actor scripts for self-modification.

Visibility

Scene manifest field

```
{
  "object": "gem",
  "id": "gem_secret",
  "visible": false,
  "x": 100, "y": 20
}
```

Field	Type	Default	Description
<code>visible</code>	bool	<code>true</code>	Whether the object's sprite is drawn. Affects drawing only – see below.

What visibility controls (and doesn't)

Affected by `visible: false`:

- The actor's sprite is not drawn on any frame.

Not affected by `visible: false`:

- The actor still ticks (`_update(dt)` runs normally).
- The actor still moves and resolves tile collision.
- The actor is still findable via `find_by_id` and `find_by_tag`.

- The actor still participates in scene state.

`visible` is a static property set in the scene manifest. In v0 there is no `set_visible()` Lua call – an actor that starts invisible stays invisible. Use a `visible: false` actor as a silent trigger zone or invisible platform, or as a placeholder for a feature you plan to reveal in a future firmware version.

TIP

For a "hidden then revealed" game mechanic in v0, use two objects: a `visible: false` trigger actor and a `visible: true` gem actor. The trigger's `_update` can call `goto_scene` or set a `state` flag to control game flow.

TurtleStudio

- The scene editor shows instance id and tags as editable fields in the object inspector.
- Objects with `visible: false` are shown at 50% opacity in the canvas preview and marked with a hidden-eye icon.
- `find_by_id` / `find_by_tag` results are reflected in the Play simulator.

Out of scope in v0

- `set_visible(handle, bool)` – runtime visibility toggle from Lua (planned for v1)
- Actor destruction from outside the actor's own script
- Inter-actor messaging

Lua API Overview

TurtleReader runs two independent Lua VMs that do not share state. Each has a distinct lifecycle and API surface.

The two VMs

	ENTRY VM	Actor VM
When it runs	Once during <code>setup()</code> at boot	Every frame via <code>_update(dt)</code>
Source	Embedded in <code>main.turtlecart</code>	<code>/scripts/<stem>.lua</code> on microSD
Lifecycle	Created → runs → destroyed	Created per scene actor, persists until scene change
Graphics	<code>cls</code> , <code>pix</code> , <code>spix</code> , <code>flip</code>	x (no direct drawing in v0)
Movement	x	<code>move</code> , <code>posx</code> , <code>posy</code> , <code>on_ground</code>
Input	<code>btn</code> , <code>btnp</code> (limited)	<code>btn</code> , <code>btnp</code> , <code>axis</code>
Animation	x	<code>set_anim</code> , <code>play_anim</code> , <code>flip_h</code>

Shared API

Both VMs have:

- `print(...)` – outputs tab-separated args to Serial (115200 baud)
- `btn(i)` / `btnp(i)` – button state (**button indices**)

Boot order

```
Mount SD
└─ Load main.turtlecart + bundle JSON
   └─ Apply PALETTE:
      └─ Run ENTRY Lua (once)
         └─ turtle_scene_begin_runtime
            ├── Draw background + tiles
            ├── Create actors, load actor scripts from SD
            └─ flip() → first frame visible
               └─ Per-frame loop:
                  ├── turtle_input_poll()
                  ├── actor _update(dt) × N
                  ├── C++ sprite/animation tick
                  └─ turtle_gpu_flip()
```

Lua standard library

`lua_openlibs` is available in both VMs, but only the documented API is guaranteed portable across cart builds.

Pages in this section

- [ENTRY VM](#) – boot script API: graphics, constants, input
- [Object Scripts](#) – per-actor `_update(dt)` API
- [Animation](#) – `set_anim`, `play_anim`, `flip_h`
- [Physics](#) – `move`, `on_ground`, gravity patterns
- [Input](#) – button indices and axis helper



ENTRY VM (v0)

The ENTRY script is declared in the `ENTRY:` line of the cartridge (TurtleStudio convention: `scripts/global.lua`). The firmware executes it once during `setup()`, before the scene runtime starts.

Constants

Name	Value	Meaning
<code>W</code>	<code>164</code>	Logical framebuffer width
<code>H</code>	<code>124</code>	Logical framebuffer height
<code>COLORS</code>	<code>32</code>	Valid color indices <code>0..31</code>

Graphics API

Color indices are integers `0..31`. Out-of-range values are clamped (`< 0` → `0`, `>= 32` → `31`). No alpha in v0.

```
cls(color_index)
```

Fills the entire framebuffer with the given palette index.

```
pix(x, y, color_index)
```

Draws a pixel in framebuffer (raster) coordinates:

- `(0, 0)` = top-left
- Y increases downward
- `x` ∈ `0..W-1`, `y` ∈ `0..H-1`
- Out-of-range coordinates are silently ignored

Use `pix` for raw screen-space operations. For game content that must align with the scene and `move()`, use `spix`.

```
spix(sx, sy, color_index)
```

Draws a pixel in scene space (same convention as object positions):

- `(0, 0)` = bottom-left
- Y increases upward
- `sx` ∈ `0..163`, `sy` ∈ `0..123`

Internal conversion: `yfb = (H - 1) - sy`, `xfb = sx`.

```
flip()
```

Copies the framebuffer to the display (if `TURTLE_USE_DISPLAY` is enabled). No arguments.

WARNING

After ENTRY, the firmware calls `flip()` again when the scene runtime finishes drawing. Any content drawn in ENTRY will be **overwritten** by the scene – ENTRY drawing only persists for carts without a bundle/scene.

Input

Input state is available but `turtle_input_poll()` hasn't been called before ENTRY runs (it runs only in `loop()`). `btn`/`btnp` will typically read all-released. Don't rely on button state in ENTRY for gameplay.

Function	Description
<code>btn(i)</code>	<code>true</code> if button <code>i</code> is held
<code>btnp(i)</code>	<code>true</code> on the press edge

`axis()` is not available in ENTRY (actor scripts only).

See [Input](#) for button index reference.

Palette

If the cart includes a `PALETTE:` block, colors are applied **before** ENTRY runs. Indices in `cls` / `pix` / `spix` refer to that table, not raw RGB.

Debug

`print(...)`

Prints tab-separated arguments followed by a newline to `Serial` (115200 baud).

Minimal example

```
-- scripts/global.lua embedded in main.turtlecart
print("TurtleReader boot")
cls(1)
flip()
```

With `INITIAL_SCENE:intro` and a bundle on the SD card, the `intro` scene overrides this drawing at step 6 of boot.

Drawing in scene space

```
cls(0)
for sx = 10, 50 do
  spix(sx, 20, 7) -- horizontal line at scene y=20
end
flip()
```

Out of scope in v0

- Game loop in Lua (the VM is destroyed after the script ends)
- `move`, `posx`, `posy`, `axis`
- Scene changes or sprite control
- Audio, text rendering, tile tables

Object Scripts (v0)

Object scripts provide per-instance game logic: reading input, moving, and changing animations. Sprite drawing and the frame loop stay in C++ (`turtle_scene`); Lua decides behavior and position.

Linking a script to an object

In the object's JSON (`objects/<id>.json` on the SD card):

```
{
  "id": "character",
  "sprite_id": "character_idle",
  "script": "character"
}
```

- `script` – stem (no `.lua` extension). Optional. If absent, the actor has no Lua tick.
- File loaded from: `/scripts/<stem>.lua` on microSD
- Valid stem: starts with a letter, then letters/digits/`_`/`-`, max 64 chars

Script contract

The runtime executes the whole file (local definitions, etc.) then calls `_update(dt)` every frame:

```
function _update(dt)
  -- dt: seconds since previous frame (e.g. ~0.033 at 30 FPS)
end
```

If `_update` is missing or the file isn't on the SD card, a warning is printed to Serial and the actor receives no Lua tick.

Implicit context

There is no `self` in `v0`. All movement/position functions act on the actor whose script is currently running. Multiple objects can share the same script stem – they get separate instances with separate positions.

Input

Function	Description
<code>btn(i)</code>	<code>true</code> if button <code>i</code> is held (sustained)
<code>btnp(i)</code>	<code>true</code> only on the frame it was first pressed
<code>axis(neg, pos)</code>	Returns -1, 0, or 1: subtracts 1 if <code>btn(neg)</code> , adds 1 if <code>btn(pos)</code>

```
local dx = axis(0, 1)  -- BTN_LEFT=0, BTN_RIGHT=1 → -1, 0, or 1
local dy = axis(3, 2)  -- BTN_DOWN=3, BTN_UP=2
```

See [Input](#) for all button indices.

Position & movement

Coordinates are in scene space (164×124, origin bottom-left, Y up).

Function	Description
<code>posx()</code>	Actor's current X position (integer)
<code>posy()</code>	Actor's current Y position (integer)

Function	Description
<code>move(dx, dy)</code>	Move with per-axis tile collision. Returns <code>ax, ay</code> – pixels actually moved.
<code>on_ground()</code>	<code>true</code> if the actor is resting on a solid tile or the scene floor

See [Physics](#) for `move` details, gravity, and jump patterns.

Animation

Animations are defined in the object's `animations` JSON array.

Function	Description
<code>set_anim(name)</code>	Switch to named animation, loop, speed 1. Does not restart if already playing.
<code>play_anim(name, speed, repeat)</code>	Switch + set speed (float) + set repeat (bool). Always restarts at frame 0.
<code>flip_h(bool)</code>	Mirror sprite horizontally (<code>true</code> = face left).

See [Animation](#) for full details.

Scene transitions

Function	Description
<code>goto_scene(scene_id)</code>	Request a scene change. Deferred – takes effect after the current frame ends. Calling it multiple times in one frame keeps only the last request.

NOTE

The ENTRY VM does **not** re-run on scene changes – it runs once at cart boot only.

Querying other actors

These functions are **read-only** in v0 – you can't move or animate another actor.

Function	Description
<code>self_id()</code>	Instance id of the actor currently running
<code>find_by_id(id)</code>	Handle (int) of the actor with that instance id, or <code>nil</code>
<code>find_by_tag(tag)</code>	Array of handles for all actors with that tag (1-based, may be empty)
<code>obj_posx(handle)</code>	X of actor at <code>handle</code> , or <code>nil</code> if invalid
<code>obj_posy(handle)</code>	Y of actor at <code>handle</code> , or <code>nil</code> if invalid
<code>obj_id(handle)</code>	Instance id of actor at <code>handle</code> , or <code>nil</code>
<code>obj_has_tag(handle, tag)</code>	<code>true</code> if actor at <code>handle</code> has the tag

WARNING

Handles are invalidated on scene change – never cache them across `goto_scene`.

```
-- Enemy approaching the player
function _update(dt)
  local player = find_by_id("player")
  if player then
    local dx = obj_posx(player) - posx()
    move((dx > 0 and 1 or -1) * 20 * dt, 0)
```

```
end
end
```

Scene script VM

A scene can declare `"script": "<stem>"` in its JSON. That script runs as a **scene VM** – it ticks before all actors each frame, with no actor attached. Useful for scene-level logic (e.g. listening for a button to transition scenes):

```
-- scripts/intro.lua
function _update(dt)
    if btnp(4) or btnp(5) then -- A or B
        goto_scene("Lvl_1")
    end
end
end
```

Platformer example skeleton

```
local walk_speed = 85
local jump_speed = 240
local gravity = 420
local vy = 0

function _update(dt)
    local dx = axis(0, 1) -- LEFT / RIGHT

    -- Horizontal movement
    move(dx * walk_speed * dt, 0)
    flip_h(dx < 0)

    -- Gravity and jump
    if on_ground() then
        if vy < 0 then vy = 0 end
        if btnp(4) then vy = jump_speed end -- BTN_A
    else
        vy = vy - gravity * dt
    end
    move(0, math.floor(vy * dt + (vy >= 0 and 0.5 or -0.5)))
end
```



```
-- Animation
if not on_ground() then
    set_anim(vy > 0 and "jump" or "fall")
elseif dx ~= 0 then
    set_anim("walk")
else
    set_anim("idle")
end
end
```

Debug

Function	Description
<code>print(...)</code>	Output to Serial (same as ENTRY)

Out of scope in v0

- `cls`, `pix`, `spix`, `flip` (ENTRY-only in v0)
- Actor-to-actor collision
- Remotely controlling another actor (handles are read-only)

Animation (v0)

Sprite animation is controlled from Lua via the `animations` list defined in the object's JSON. C++ advances frames automatically after `_update(dt)` runs.

Defining animations in the object JSON

```
{
  "id": "character",
  "animations": [
    { "name": "idle", "sprite_id": "character_idle" },
    { "name": "walk", "sprite_id": "character_walk" },
    { "name": "jump", "sprite_id": "character_jump" },
    { "name": "fall", "sprite_id": "character_fall" }
  ]
}
```

- `name` – logical identifier used in Lua
- `sprite_id` – stem of the sprite file (`sprites/<stem>.tsp`)

API

Function	Description
<code>set_anim(name)</code>	Switch to <code>name</code> . Loops infinitely at speed 1. Does not restart if already playing that animation.
<code>play_anim(name, speed, repeat)</code>	Switch to <code>name</code> . Always restarts at frame 0. <code>speed</code> is a float; <code>repeat</code> is a boolean.
<code>flip_h(bool)</code>	Horizontal mirror (<code>true</code> = face left). Flips around the sprite anchor in scene space.

Frame rate

`speed = 1.0` → one frame every `1000 / default_anim_fps` ms (e.g. 8 FPS → 125 ms/frame).

`speed = 2.0` → twice as fast. Valid range: `0.25..16`.

Repeat

- `true` – loops back to frame 0 after the last frame
- `false` – holds on the last frame (useful for `damage`, cutscenes)

Performance

- `set_anim` is a no-op if the actor is already on that animation – safe to call every frame.
- Sprites are pre-loaded into PSRAM at scene start (up to 48 blobs per `sprite_id`). Frame advances read from RAM, not SD.
- Changing animation invalidates the actor's pixel cache; C++ redraws it on the next frame.

Example

```
if not on_ground() then
  if vy > 0 then
    set_anim("jump")
  else
    set_anim("fall")
  end
elseif dx ~= 0 then
  set_anim("walk")
  flip_h(dx < 0)
else
  set_anim("idle")
end

-- One-shot damage animation (slower, no loop):
-- play_anim("damage", 0.75, false)
```

Notes

- `move()` no longer affects animation speed (it did in early firmware). Lua has full control.
- Frame advance is performed by C++ in `tick_actors`, after `_update` returns.
- If `name` doesn't exist in the object's animation list, the firmware logs a warning to Serial and keeps the current sprite.

Physics (v0)

Movement and collision for actors in scene. Inspired by Godot's `move_and_collide` – per-axis, no slope sliding.

Responsibility split

Layer	Handles
Lua	Velocity (<code>vx</code> , <code>vy</code>), gravity, jump impulse, input
C++	AABB vs solid tiles, scene bounds, <code>on_ground()</code> flag

`move(dx, dy)` → `ax, ay`

Moves the actor by `(dx, dy)` pixels with tile collision:

1. Resolves X in 1-px steps
2. Resolves Y in 1-px steps
3. If moving down (`dy < 0` in scene space) and it hits a tile → sets **grounded**
4. Clamps to scene bounds using the actor's collision AABB

Returns `ax, ay` – the pixels **actually moved**, which may be less than requested if a tile blocks the path.

`on_ground()` → **bool**

`true` if the actor is resting on a solid tile just below its AABB, or at scene floor `y = 0`.

Reflects state after the last `move()` in the current frame.

Collision AABB

Defined in the object JSON, field `collision`:

```
"collision": {  
  "mode": "aabb",  
  "x0": -7,  
  "y0": 0,  
  "x1": 6,  
  "y1": 16  
}
```

Coordinates are local to the sprite anchor (feet at `(0, 0)`, Y up). If absent, the firmware derives a box from the sprite size and `origin`.

Gravity pattern (Y up = positive)

```
if on_ground() then  
  vy = 0  
else  
  vy = vy - gravity * dt -- falling: vy decreases (goes negative)  
end  
move(0, math.floor(vy * dt + (vy >= 0 and 0.5 or -0.5)))
```

Jump pattern

```
local jump_speed = 240 -- px/s upward  
local gravity    = 420 -- px/s2  
  
if on_ground() then  
  if vy < 0 then vy = 0 end -- reset downward velocity on landing  
  if btnp(4) then          -- BTN_A  
    vy = jump_speed  
  end  
else  
  vy = vy - gravity * dt  
end
```

Jump height $\approx \frac{\text{jump_speed}^2}{2 \times \text{gravity}}$ – with the values above: ~69 px.

Common bugs and fixes

Bug 1 – Stuck on ceiling after jump

Cause: `ay < my` when hitting a ceiling, but `vy` isn't reset so the actor keeps requesting upward motion.

Fix:

```
local my = math.floor(vy * dt + (vy >= 0 and 0.5 or -0.5))
local ax, ay = move(0, my)
if my > 0 and ay < my then
  vy = 0 -- hit ceiling; cut upward impulse immediately
end
```

Bug 2 – Teleport when un-sticking from a wall

Cause: horizontal remainder accumulates while blocked, then fires all at once when the block clears.

Fix: discard the remainder on block:

```
local mx = math.floor(rem_x + 0.5)
local ax, ay = move(mx, 0)

if ax == mx then
  rem_x = rem_x - ax -- applied, keep sub-pixel remainder
else
  rem_x = 0.0 -- blocked: discard, don't let it accumulate
end
```

Full platformer example

```
local walk_speed = 85
```

```

local jump_speed = 240
local gravity    = 420
local vy        = 0
local rem_x      = 0.0

function _update(dt)
    local dx = axis(0, 1)  -- LEFT, RIGHT

    -- Horizontal with sub-pixel accumulator
    rem_x = rem_x + dx * walk_speed * dt
    local mx = math.floor(rem_x + 0.5)
    local ax = select(1, move(mx, 0))
    if ax == mx then rem_x = rem_x - ax else rem_x = 0.0 end
    flip_h(dx < 0)

    -- Gravity + jump
    if on_ground() then
        if vy < 0 then vy = 0 end
        if btnp(4) then vy = jump_speed end
    else
        vy = vy - gravity * dt
    end
    local my = math.floor(vy * dt + (vy >= 0 and 0.5 or -0.5))
    local _, ay = move(0, my)
    if my > 0 and ay < my then vy = 0 end  -- ceiling hit

    -- Animation
    if not on_ground() then
        set_anim(vy > 0 and "jump" or "fall")
    elseif dx ~= 0 then
        set_anim("walk")
    else
        set_anim("idle")
    end
end
end

```

Out of scope in v0

- `move_and_slide` / surface normals
- One-way platforms (supported at tile level but not exposed to Lua separately)
- Actor-to-actor collision

Input (v0)

Button indices

8 buttons are supported in v0. Typical wiring: one side to a GPIO, other to GND. Firmware uses `INPUT_PULLUP` (released = HIGH, pressed = LOW).

Index	Name	Common use
<code>0</code>	LEFT	Move left
<code>1</code>	RIGHT	Move right
<code>2</code>	UP	Move up / menu up
<code>3</code>	DOWN	Move down / menu down
<code>4</code>	A	Jump / confirm
<code>5</code>	B	Action 2 / cancel
<code>6</code>	C	Action 3
<code>7</code>	D	Action 4

Reserved for future expansion to 11 buttons (menu, start, etc.).

API

`btn(i)` → **bool**

`true` if button `i` is currently held down.

`btnp(i)` → **bool**

`true` only on the frame it was first pressed (rising edge). The firmware retains the edge until Lua reads it – so even at 30 FPS you won't miss a fast tap.

`axis(neg, pos)` → **int** *(actor scripts only)*

Convenience helper: returns -1 if `btn(neg)`, +1 if `btn(pos)`, 0 otherwise.

```
local dx = axis(0, 1) -- -1=left, 0=neutral, +1=right
local dy = axis(3, 2) -- -1=down (scene Y down), +1=up
```

`axis` is not available in the ENTRY VM.

WARNING

An invalid button index (`< 0` or `> 7`) causes a Lua error (unlike color clamping which is silent).

Where input is available

Context	Available
ENTRY VM	<code>btn</code> , <code>btnp</code> – but <code>turtle_input_poll()</code> hasn't run yet; state is usually all-released
Actor scripts <code>_update(dt)</code>	<code>btn</code> , <code>btnp</code> , <code>axis</code> – polled every frame
Scene scripts <code>_update(dt)</code>	<code>btn</code> , <code>btnp</code> , <code>axis</code> – same as actor scripts

Pin configuration

Button GPIO pins are defined in `firmware/TurtleReader/turtle_input.h` (`TURTLE_BTN_PIN_*`). Set a pin to -1 to disable that button. Avoid pins 36-39 (used by SD) and 8-12 (used by the display in the default config).



Scene Script (v0)

A scene script is an optional Lua file that runs inside the Actor VM alongside all actor scripts. It is ideal for scene-level coordination: checking conditions that span multiple actors, triggering transitions, managing global game state within a scene.

Declaration

Add a `"script"` field to the scene object in the bundle:

```
{
  "id": "level_1",
  "script": "level1",
  ...
}
```

The firmware loads `/scripts/level1.lua` from the SD card at scene start. If the file is missing, the firmware logs a Serial warning and the scene runs without a scene script.

Execution model

The scene script runs in the same `lua_State` as all actor scripts. It shares the Lua global table with actors.

Tick order per frame:

1. Scene script `_update(dt)` – called **before** any actor's `_update(dt)`
2. All actor `_update(dt)` calls (in manifest order)

This ordering means the scene script can set globals that actors read within the same frame.

Available API

The scene script has access to the same actor VM bindings – minus the self-referential actor functions:

Available	Not available
<code>btn(i)</code> / <code>btnp(i)</code> / <code>axis(i)</code>	<code>posx()</code> / <code>posy()</code> (no self)
<code>goto_scene(id)</code>	<code>move(dx, dy)</code> / <code>on_ground()</code>
<code>find_by_id(id)</code> / <code>find_by_tag(tag)</code>	<code>set_anim</code> / <code>play_anim</code> / <code>flip_h</code>
<code>obj_posx/posy/id/has_tag(handle)</code>	<code>text(str, ...)</code> as actor overlay
<code>state_get(key)</code> / <code>state_set(key, val)</code> / <code>state_add(key, delta)</code>	–
<code>print(...)</code>	–

Naming convention

Because scene script globals live in the same namespace as all actor globals, use a consistent prefix to avoid collisions:

```
-- levell1.lua – scene script
scene_timer = 0
scene_enemies_left = 5
```

```
function _update(dt)
    scene_timer = scene_timer + dt

    if scene_enemies_left <= 0 then
        goto_scene("level_2")
    end
end
```

Actor scripts can read `scene_enemies_left` directly; an enemy actor that dies can decrement it:

```
-- enemy.lua - actor script
function _update(dt)
    -- ... enemy logic ...
    if hp <= 0 then
        scene_enemies_left = scene_enemies_left - 1
        -- self-destruct or freeze
    end
end
```

WARNING

Global names beginning with `_` are reserved for firmware lifecycle hooks (`_update`, `_hud`, `_hud_init`). Use alphabetic prefixes like `scene_` or `g_` for scene-level globals.

Common patterns

Scene timer / countdown

```
-- scene script
scene_countdown = 60 -- seconds

function _update(dt)
    scene_countdown = scene_countdown - dt
    if scene_countdown <= 0 then
        goto_scene("game_over")
    end
end
```

```
end
end
```

Boss gate

```
-- scene script
function _update(dt)
  local boss = find_by_id("boss")
  if boss == nil then
    -- boss was destroyed; transition after a brief pause
    scene_win_timer = (scene_win_timer or 0) + dt
    if scene_win_timer > 2.0 then
      goto_scene("credits")
    end
  end
end
end
```

Persistent progress via state store

```
-- scene script: save checkpoint on reaching a flag
function _update(dt)
  local flag = find_by_id("checkpoint_1")
  local player = find_by_tag("player")[1]
  if flag and player then
    local dx = obj_posx(player) - obj_posx(flag)
    if math.abs(dx) < 8 then
      state_set("last_checkpoint", 1)
    end
  end
end
end
```

Interaction with `_hud` (ENTRY VM)

The scene script runs in the Actor VM; `_hud(dt)` runs in the ENTRY VM. They cannot call each other directly. Coordination goes through the **state store**: the scene script writes `state_set("score", n)`, and `_hud(dt)` reads `state_get("score")` to display it.

Actor writes to state are visible to ENTRY's `_hud(dt)` within the **same frame** because the frame ordering is:

1. Scene script `_update` → actor `_update` calls (Actor VM ticks)
 2. `_hud(dt)` (ENTRY VM tick)
 3. GUI layers
 4. `flip()`
-

TurtleStudio

- The scene inspector has a "Scene script" field accepting a bare stem (no `.lua` extension).
- The Lua editor tab shows the scene script file alongside actor scripts.
- The Play simulator loads the scene script from the project's `scripts/` directory.



State Store (v0)

The state store is a small key-value memory shared across all Lua VMs (ENTRY and all actor scripts). It is the canonical way for actors to communicate with the HUD, to persist progress between scenes, and to coordinate between actor scripts.

Characteristics

Property	Value
Slots	16 simultaneous keys
Key type	String, max 31 chars
Value type	<code>int32</code> (signed 32-bit integer)
Lifetime	Persists across scene transitions; reset to all-zero on cart load
Visibility	Every VM sees writes immediately within the same frame

The 16-slot limit is per cart. Choose short, descriptive keys – they are the only documentation of what a slot means.

API

```
state_set(key, value)
```

Write `value` (integer) to `key`. Creates the slot if it doesn't exist; overwrites if it does.

```
state_set("score", 0)
state_set("lives", 3)
```

Returns nothing. If the store is full (16 keys) and `key` is new, the write is a silent no-op – check Serial output during development.

`state_get(key)` → `int | nil`

Read the current value for `key`. Returns `nil` if the key has never been written.

```
local score = state_get("score") or 0
```

Use `or 0` (or another default) to guard against the first-run `nil`.

`state_add(key, delta)` → `int`

Atomically add `delta` to the value at `key` and return the new value. If `key` doesn't exist, it is treated as `0` before the add.

```
local new_score = state_add("score", 100)
```

Equivalent to `state_set(key, (state_get(key) or 0) + delta)`, but in a single call. Useful for counters that multiple actors may increment independently.

Cross-VM visibility

State writes from actor scripts are visible to ENTRY's `_hud(dt)` within the same frame, because the per-frame order is:

1. Actor VM: scene script `_update(dt)` → actor `_update(dt)` calls
2. ENTRY VM: `_hud(dt)`

3. GUI layers

4. `flip()`

An actor that calls `state_add("score", 100)` in step 1 will have the HUD read the new value in step 2, on the same frame. No one-frame lag.

Common patterns

Score counter

```
-- actor script (e.g. coin.lua)
function _update(dt)
    local px = find_by_tag("player")[1]
    if px and math.abs(obj_posx(px) - posx()) < 8 then
        state_add("score", 10)
        goto_scene("level_1") -- respawn / collect
    end
end
```

```
-- ENTRY VM (_hud function)
function _hud(dt)
    hud_clear(0)
    hud_text(4, 4, "SCORE " .. (state_get("score") or 0), "font8")
end
```

Lives / retry loop

```
-- scene script
function _update(dt)
    if state_get("lives") ~= nil and state_get("lives") <= 0 then
        goto_scene("game_over")
    end
end
```

```
-- player actor script, on death:
state_add("lives", -1)
```

```
goto_scene("level_1") -- restart
```

Cross-scene persistent flag

```
-- level_1 scene script
state_set("boss_defeated", 0)

-- boss actor, on death:
state_set("boss_defeated", 1)

-- level_2 scene script (read flag set in level_1):
function _update(dt)
  if state_get("boss_defeated") == 1 then
    -- skip intro cutscene
  end
end
```

Key naming guidelines

- Use lowercase `snake_case`: `"score"`, `"lives"`, `"gears_collected"`.
- Stay under 16 total keys for the whole cart – treat the store as a tight global register file, not a database.
- Document your key names in the ENTRY script or a comment in your scene script – there is no introspection API.

Limits and edge cases

Situation	Behavior
<code>state_get</code> on an unset key	Returns <code>nil</code>
<code>state_add</code> on an unset key	Treats existing value as <code>0</code> , creates key
17th unique key written	Silent no-op; value is not stored

Situation	Behavior
Integer overflow (exceeding int32 range)	Wraps (C signed overflow semantics)
Scene change	State is preserved
Cart reload / power cycle	State is reset to zero

Firmware-Lua Bridge (v0)

This page documents how the C++ firmware exposes functionality to the two Lua VMs, what each VM can and cannot do, and where the code lives for each binding.

Two independent VMs

The firmware creates **two completely separate** Lua states. They never share a `lua_State`, never share globals, and cannot call each other directly.

Property	ENTRY VM	Actor VM
<code>lua_State</code>	Dedicated	Shared across all actors in the scene
Lifetime	Entire cart execution	Created at scene start, destroyed at scene end
Ticked by	<code>TurtleReader.ino</code> <code>setup()</code>	<code>turtle_scene_runtime_tick</code> each frame
Script source	<code>ENTRY:<path></code> in <code>.turtlecart</code>	<code>objects/<id>.json</code> → "script"
Drawing	Immediate (ENTRY globals, HUD)	Via actor overlay system
VM isolation	Runs first, completes before scene starts	Runs per-frame inside scene loop

Communication between VMs goes through the **state store**

(`state_get` / `state_set` / `state_add`) – 16 int32 slots that both VMs can read and write. See [State Store](#).

API surface by VM

ENTRY VM bindings

Bound in `TurtleReader.ino` and `turtle_gpu.cpp`.

Function	Description
<code>cls(i)</code>	Fill framebuffer with palette index <code>i</code> .
<code>pix(xfb, yfb, i)</code>	Draw pixel at raw framebuffer coords (Y-down).
<code>spix(sx, sy, i)</code>	Draw pixel at scene coords (origin bottom-left, Y-up).
<code>flip()</code>	Push framebuffer to display.
<code>text(sx, sy, str, font_id [, color])</code>	Draw text immediately at scene coords.
<code>text_width(str, font_id)</code>	Measure text width in pixels without drawing.
<code>btn(i)</code> / <code>btnp(i)</code>	Read button state / pressed-this-frame.
<code>W</code> , <code>H</code> , <code>COLORS</code>	Constants: 164, 124, 32.
<code>hud_pix</code> , <code>hud_rect</code> , <code>hud_clear</code> , <code>hud_text</code> , <code>hud_text_width</code>	HUD-strip drawing (see HUD Border).
<code>gui_layer_show</code> , <code>gui_layer_hide</code> , <code>gui_layer_set_text</code> , ...	GUI layer control (see GUI Layers).
<code>state_get</code> , <code>state_set</code> , <code>state_add</code>	Cross-VM state store (see State Store).

`text()` in the ENTRY VM draws **immediately** at absolute scene coordinates. It is wiped as soon as the scene starts (`turtle_scene_begin_runtime` calls `cls`). Use it for splash screens and boot animations; for persistent in-game text use scene `text_labels` or GUI layers.

Actor VM bindings

Bound in `turtle_actor_lua.cpp`.

Function	Description
<code>btn(i)</code> / <code>btnp(i)</code> / <code>axis(i)</code>	Input.
<code>posx()</code> / <code>posy()</code>	Actor position in scene coordinates.
<code>move(dx, dy)</code>	Move with tile collision; returns actual pixels moved.
<code>on_ground()</code>	True if grounded after the last <code>move</code> .
<code>set_anim(name)</code> / <code>play_anim(name [, speed])</code>	Animation control.
<code>flip_h(bool)</code>	Mirror sprite horizontally.
<code>text(str, font_id [, dx, dy, color])</code>	Persistent text overlay on this actor (different signature than ENTRY).
<code>text_width(str, font_id)</code>	Measure text width without drawing.
<code>goto_scene(id)</code>	Transition to a named scene.
<code>find_by_id(id)</code> / <code>find_by_tag(tag)</code>	Object identity queries.
<code>obj_posx(h)</code> / <code>obj_posy(h)</code> / <code>obj_id(h)</code> / <code>obj_has_tag(h, tag)</code>	Handle queries.

Function	Description
<code>state_get</code> , <code>state_set</code> , <code>state_add</code>	Cross-VM state store.

`text()` signature difference

The two VMs have different `text()` signatures because they have different drawing models:

ENTRY `text(sx, sy, str, font_id [, color])` – immediate blit at absolute scene position. The call draws and returns immediately. Suitable for splash screens.

Actor `text(str, font_id [, dx, dy, color])` – a setter that attaches a text overlay to the actor. The firmware redraws this overlay at the actor's current position every frame as part of the actor draw pass. `dx` / `dy` offset the overlay relative to the actor's anchor. Call with `""` to clear. This design is required for correct dirty-rect erasure: if the actor moves, the firmware knows exactly which cells to clear and repaint on the next frame.

Font cache

Both VMs share a single font cache (max 4 fonts simultaneously). A font is loaded from the bundle's `.tfn` assets the first time it is referenced and evicted by LRU if the cache is full. Keep the number of distinct fonts per scene to 4 or fewer to avoid repeated SD loads.

Data origins

Data	Source	Notes
ENTRY Lua script	Embedded in <code>.turtlecart</code>	Loaded into RAM at cart open
Actor Lua scripts	<code>/scripts/<stem>.lua</code> on SD	Loaded at scene start
Bundle JSON (sprites, objects, scenes, guilayers)	<code>/studio/project_bundle.json</code> on SD	Parsed at cart open
Sidecar binary assets (<code>.tbg</code> , <code>.tsp</code> , <code>.tts</code> , <code>.tfn</code>)	SD card	Loaded on demand per scene
Palette	<code>PALETTE:</code> block in <code>.turtlecart</code>	Applied once at cart open
State store	RAM	Zeroed at cart load; persists across scenes

Firmware source map

File	Role
<code>TurtleReader.ino</code>	ENTRY VM setup, button polling, SDL loop
<code>turtle_gpu.cpp/.h</code>	Framebuffer primitives bound to ENTRY
<code>turtle_actor_lua.cpp/.h</code>	Actor VM lifecycle, all actor bindings
<code>turtle_scene.cpp/.h</code>	Scene runtime, actor position/animation/collision, <code>text</code> overlay
<code>turtle_font.cpp/.h</code>	Font loading, measuring, drawing (shared by both VMs)
<code>turtle_input.cpp/.h</code>	GPIO polling; <code>btn</code> / <code>btnp</code> state shared to both VMs



Assets



Sprite Spec

Sprite Spec (v0)

Sprites in TurtleStudio are JSON assets stored in `objects/Sprites/<id>.json`. They define pixel art with optional multiple frames, a palette reference, an anchor origin, and a collision shape when attached to an object.

Size: cells and pixels

Field	Meaning
<code>cell_px</code>	Side of one design cell in pixels (integer <code>8..256</code> , multiple of 8). Default: 8.
<code>blocks_w</code> , <code>blocks_h</code>	Sprite size in cells (<code>1..32</code> per axis).
<code>pixel_w</code> , <code>pixel_h</code>	Size in pixels: <code>blocks_w * cell_px</code> , <code>blocks_h * cell_px</code> .

The firmware always draws using `pixel_w * pixel_h`. `cell_px` is purely a grid/editor convention – two sprites can use different `cell_px` values. Sprites with legacy `cell_px: 4` remain valid; the editor only rewrites it when you explicitly use Resize.

The 8 px cell default matches the Game Boy / Game Gear VRAM tile convention.

Sprite JSON format

```
{
```

```

"format_version": 1,
"kind": "turtlestudio.sprite",
"id": "hero",
"notes": "",
"palette": "palettes/palette.txt",
"cell_px": 8,
"blocks_w": 2,
"blocks_h": 3,
"pixel_w": 16,
"pixel_h": 24,
"origin_x": 8,
"origin_y": 0,
"render": { "mode": "indexed_pixels" },
"image": {
  "format": "palette_rows",
  "rows": [
    [31, 31, 0, 1, 0, 31, 31, 31, 0, 31, 31, 31, 0, 1, 0, 31],
    ...
  ]
},
"frame_count": 3,
"frames": [
  { "image": { "format": "palette_rows", "rows": [...] } },
  { "image": { "format": "palette_rows", "rows": [...] } }
]
}

```

Field	Description
<code>palette</code>	Project-relative path to a <code>#RRGGBB</code> -per-line palette file. Pixel indices in <code>image.rows</code> refer to this palette.
<code>frame_count</code>	Total frame count, <code>1..32</code> . Default <code>1</code> .
<code>image</code>	Frame 0 pixel data (always present for firmware compatibility).
<code>frames</code>	Frames 1..N-1. Each entry is <code>{ "image": { "format": "palette_rows", "rows": [...] } }</code> .

Render modes

`solid_palette_index`

A solid rectangle filled with a single palette index. No `image` data needed.

```
"render": { "mode": "solid_palette_index", "palette_index": 4 },
"image": null
```

`indexed_pixels` *(recommended)*

A 2D array of palette indices. Row `0` is the **top** of the sprite (same row-0-top convention as the editor).

```
"render": { "mode": "indexed_pixels" },
"image": {
  "format": "palette_rows",
  "rows": [
    [31, 31, 0, 1],
    [31, 0, 2, 1]
  ]
}
```

- Each row has `pixel_w` integers in range `0..31`.
- Index 31 is always **transparent** – pixels with this index are not drawn. Use it for the sprite's "holes."
- Incomplete rows are padded by tools on normalize/save.

The export pipeline also accepts `"palette_rows_rle"` (runs of `[index, count]`) for compactness; the firmware understands both.

Origin (anchor point)

`origin_x` and `origin_y` define the sprite's **anchor** in sprite-local coordinates:

- `(0, 0)` = **bottom-left** corner of the `pixel_w × pixel_h` bounding box

- Y increases upward (same as scene space)
- Range: `0..pixel_w-1` for X, `0..pixel_h-1` for Y
- Default: `(0, 0)` – bottom-left is the anchor

When a scene places an object at `(x, y)`, the anchor lands exactly at that point. The top-left corner of the bounding box is drawn at `(x - origin_x, y - origin_y)`.

```
origin_x=8, origin_y=0 on a 16×24 sprite:  
→ anchor at the horizontal center, feet level  
→ bbox drawn at (object_x - 8, object_y)
```

TurtleStudio shows the anchor as a magenta crosshair on the sprite canvas.

Object JSON

Objects are the game entities placed in scenes. Each object references one or more sprites via `sprite_id` and `animations`.

```
{  
  "format_version": 1,  
  "kind": "turtlestudio.object",  
  "id": "character",  
  "name": "character",  
  "sprite_id": "character_idle",  
  "script": "character",  
  "animations": [  
    { "name": "idle", "sprite_id": "character_idle" },  
    { "name": "walk", "sprite_id": "character_walk" },  
    { "name": "jump", "sprite_id": "character_jump" }  
  ],  
  "collision": {  
    "mode": "aabb",  
    "x0": -7, "y0": 0,  
    "x1": 6, "y1": 23  
  }  
}
```



```
}
```

Field	Description
<code>sprite_id</code>	Default sprite (rest pose, scene placement).
<code>script</code>	Stem of <code>scripts/<stem>.lua</code> – the actor Lua script. Optional.
<code>animations</code>	List of <code>{ "name", "sprite_id" }</code> . Names must start with a letter, then letters/digits/ <code>_</code> / <code>-</code> , max 32 chars. Max 32 animations per object.
<code>collision</code>	Collision shape in anchor-local space (see below). Optional.

Collision shapes

Coordinates are local to the anchor (`(0, 0)` = origin of the sprite in scene space, Y up).

AABB (rectangle):

```
"collision": {  
  "mode": "aabb",  
  "x0": -7, "y0": 0,  
  "x1": 6,  "y1": 23  
}
```

`x0, y0` = bottom-left corner (inclusive); `x1, y1` = top-right corner.

Triangle:

```
"collision": {  
  "mode": "triangle",  
  "points": [[-7, 0], [6, 0], [0, 23]]  
}
```

3 vertices `[x, y]`.

Hexagon:

```
"collision": {  
  "mode": "hexagon",  
  "points": [[-4,0],[-7,10],[-4,20],[4,20],[7,10],[4,0]]  
}
```

6 vertices (clockwise or counter-clockwise).

NOTE

The firmware approximates `triangle` and `hexagon` as AABB (bounding box of their vertices). TurtleStudio uses the actual shape for its editor preview; only the AABB is used for collision at runtime.

If `collision` is absent, the firmware falls back to the sprite's bounding box from `origin`.

Project bundle (`studio/project_bundle.json`)

When TurtleStudio exports `main.turtlecart`, it writes (or references) sprite data in the bundle:

Field	Description
<code>transparent_index</code>	Always <code>31</code> .
<code>target_fps</code>	Game loop FPS (default <code>30</code> , range <code>15-60</code>). Per-project default; each scene can override.
<code>default_anim_fps</code>	Sprite animation FPS at <code>speed=1</code> (default <code>8</code> , range <code>1-30</code>). Per-project default; each scene can override.
<code>sprites</code>	Map of <code>id</code> → inline sprite JSON or <code>{ "kind": "turtlestudio.sprite_ref", "file": "sprites/<id>.json" }</code>

Field	Description
<code>objects</code>	Map of <code>id</code> → inline object JSON or <code>{ "kind": "turtlestudio.object_ref", "file": "objects/<id>.json" }</code>

All sprites referenced by default `sprite_id` and `animations` are embedded in the bundle at export time.

Firmware draw sequence (boot)

If `studio/project_bundle.json` exists and `INITIAL_SCENE` matches a scene in the bundle, the firmware draws the scene **before** running the ENTRY Lua:

1. `cls(background_index)` – fill with the scene's background palette index
2. Draw the background asset at scene origin `(0, 0)` (if the scene declares one)
3. For each object in the scene's `objects[]`: resolve `sprite_id`, draw at `(x - origin_x, y - origin_y)`

Draw modes

Mode	Firmware behavior
<code>solid_palette_index</code>	<code>fill_rect</code> with <code>render.palette_index</code>
<code>indexed_pixels</code>	Blit <code>image.rows</code> ; skip pixels where index = <code>transparent_index</code> (31)

Size resolution

The firmware prefers explicit `pixel_w` / `pixel_h`. If absent, it computes `blocks_w × cell_px` using `cell_px` from the JSON, or `kDefaultCellPx = 8`

if that's also missing.

Firmware limits: sprite up to 128×128 px in static RAM; background indexed image up to 164×124 px (full scene); palette indices clamped to 0..31.

TurtleStudio sprite editor

- **Palette:** click an index to select it as the paint brush. Index 31 cannot be selected as a brush.
 - **Right-click / right-drag:** erases (writes index 31).
 - **Clear all:** fills the entire canvas with index 31.
 - **Colors used:** row of indices currently present in the canvas; click to re-select.
 - **Swap color:** replace all pixels of one index with another.
 - **Frames:** `frame_count` field + F0, F1, ... tabs above the canvas. Each tab edits a separate pixel matrix. Save writes all frames to JSON.
 - **Reference image:** import a PNG/JPG scaled to `pixel_w × pixel_h`. Visible under the paint layer; **Convert to sprite** maps each pixel to the nearest palette index (alpha < 0.5 → index 31). Reference is not saved in the JSON.
 - **Resize:** rewrites `cell_px` (must be a multiple of 8). Pixels outside the new bounds are kept in memory until save.
 - **Canvas background:** preview-only color for transparent pixels; not part of the palette.
-

Compatibility notes

Case	Behavior
<code>cell_px: 4</code> or other non-multiple of 8	Valid; editor only rewrites it on explicit Resize. Runtime always uses <code>pixel_w</code> / <code>pixel_h</code> .
<code>solid_palette_index</code> sprites	Still work on hardware; saved as <code>indexed_pixels</code> after any edit.
Scene palette $\neq$ sprite palette	Allowed in data. The firmware uses the cart-level <code>PALETTE:</code> for all index→RGB lookups. Aligning palettes is the author's responsibility.

Implementation references

File	Role
<code>tools/turtlestudio/src/turtlestudio/sprites.py</code>	Sprite serialization, <code>DEFAULT_CELL_PX</code> , editor constants
<code>tools/turtlestudio/src/turtlestudio/build.py</code>	Bundle export
<code>firmware/TurtleReader/turtle_scene.cpp</code>	Scene draw, sprite resolution, <code>kDefaultCellPx</code>
<code>firmware/TurtleReader/turtle_gpu.cpp</code>	<code>turtle_gpu_blit_indexed_scene</code> – pixel blit with transparent-index skip

Out of scope in v0

- In-firmware sprite animation playback (multi-frame data is stored; playback via actor scripts using `set_anim` / `play_anim` – see [Animation](#))
- Hardware rotation / flip

- Pixel data compression in the JSON format
- Full background layer compositing at boot (v0 draws a single background + objects)

Asset Binary Formats (v0)

TurtleStudio edits assets as JSON on disk. When exporting to the SD package (`build/`), pixel data is baked into compact binary files.

Extension	Type	Magic
<code>.tbg</code>	Background image	<code>TBG\0</code>
<code>.tsp</code>	Sprite sheet	<code>TSP\0</code>
<code>.tts</code>	Tileset	<code>TTS\0</code>
<code>.tfn</code>	Font	<code>TFN\0</code>
<code>.json</code>	Object definitions	(plain text, small)

All binary formats are little-endian.

Sprite / Background common header

Applies to `.tsp` (v0) and `.tbg`:

Offset	Field	Size
0	Magic (<code>TBG\0</code> or <code>TSP\0</code>)	4 bytes
4	<code>version</code>	u8
5	<code>flags</code>	u8 (= 0)
6	<code>pixel_w</code>	u16

Offset	Field	Size
8	<code>pixel_h</code>	u16
10	<code>mode</code>	u8

Sprite multi-frame (`.tsp` version 1)

Used when a sprite has more than one frame:

Offset	Field	Size
4	<code>version</code> = 1	u8
6	<code>pixel_w</code>	u16
8	<code>pixel_h</code>	u16
10	<code>frame_count</code>	u16
12+	Per frame: <code>chunk_len</code> u32 + <code>[mode u8][payload...]</code>	variable

The exporter uses version 1 when `frame_count > 1`.

Pixel modes

Value	Name	Payload
<code>0</code>	<code>SOLID</code>	1 byte: single palette index (whole image one color)
<code>1</code>	<code>RAW</code>	<code>pixel_w * pixel_h</code> bytes (row 0 = top)
<code>2</code>	<code>ROW_RLE</code>	Per row: <code>nruns</code> u16, then <code>nruns * (idx u8, count u16)</code>

The exporter automatically chooses the smallest mode: `SOLID < RLE < RAW`.

Palette index 31 is always transparent – pixels with that index are not drawn.

Tileset (.tts)

Header (same magic convention, `TTS\0`):

Offset	Field	Size
4	<code>version</code> (0 = no collision, 1 = with collision)	u8
6	<code>tile_px</code> (tile size in pixels, square)	u16
8	<code>tile_count</code>	u16
10+	Per tile: <code>chunk_len</code> u32 + indexed blob (<code>tile_px × tile_px</code>)	variable

Collision block (version 1 only)

Immediately after the last tile chunk, `tile_count` records of 10 bytes each (1:1 with the `tiles[]` order):

Offset	Field	Size
0	<code>kind</code> : 0=solid, 1=none, 2=AABB shape	u8
1	<code>flags</code> : bit0=one-way; bits1-2=direction (0=up,1=down,2=left,3=right)	u8
2	<code>x0</code> (tile-local, Y up, only for <code>kind=2</code>)	i16 LE
4	<code>y0</code>	i16 LE

Offset	Field	Size
6	<code>x1</code>	i16 LE
8	<code>y1</code>	i16 LE

Tiles without explicit collision data export with `kind=0` (solid) – same as the firmware default.

Font (`.tfn`)

Header is 14 bytes (distinct from the sprite/bg header – fonts are N square glyphs, not a single image):

Offset	Field	Size
0	Magic <code>TFN\0</code>	4 bytes
4	<code>version</code> = 0	u8
5	<code>flags</code> = 0	u8
6	<code>glyph_px</code> (square glyph size, same for all)	u16
8	<code>line_height</code>	u16
10	<code>baseline</code>	u16
12	<code>glyph_count</code>	u16

Glyph records

Immediately after the header, `glyph_count` variable-size records:

Field	Size
<code>advance</code> (horizontal advance in px, 1..255)	u8
<code>chunk_len</code>	u32 LE
chunk blob (indexed <code>glyph_px * glyph_px</code> , same format as <code>.tsp</code>)	<code>chunk_len</code> bytes

⚠ WARNING

The `.tfn` format does **not** embed which character maps to which glyph. Glyph order matches the `charset` field of the font JSON at export time. The firmware assumes the fixed `LATIN_CHARSET` order (space, A-Z, a-z, 0-9, `.,!?:;'-`). If the editor ever supports custom charsets, the format must bump to version 1 with an embedded charset.

Glyphs absent from the JSON are exported with solid fill (palette index 1), not transparent.

Bundle manifest (`studio/project_bundle.json`)

The bundle references baked assets by path:

```
{
  "backgrounds": {
    "sky": { "kind": "turtlestudio.background_ref", "file":
"backgrounds/sky.tbg" }
  },
  "fonts": {
    "default": { "kind": "turtlestudio.font_ref", "file":
"fonts/default.tfn" }
  }
}
```

Implementation references

File	Role
<code>firmware/TurtleReader/turtle_asset_bin.cpp</code>	Decoder (<code>.tsp</code> , <code>.tbg</code>)
<code>firmware/TurtleReader/turtle_tileset.cpp</code>	Tileset decoder (<code>.tts</code>)
<code>firmware/TurtleReader/turtle_font.cpp</code>	Font decoder (<code>.tfn</code>)
<code>tools/turtlestudio/src/turtlestudio/asset_bin.py</code>	Encoder (export pipeline)
<code>firmware/host_tests/run_font_test.sh</code>	Round-trip verification for <code>.tfn</code>

[GUI](#)[HUD Border](#)

HUD Border (v0)

The HUD border reserves fixed pixel strips along the framebuffer edges for permanent on-screen UI – score, lives, minimap, timer – by shrinking the camera's playfield. This is **method 1** of GUI: always-visible during gameplay, zero per-frame cost when static. [GUI layers](#) are **method 2**: transient overlays for menus, pause screens, and dialogues.

The two methods are independent and compose cleanly – a GUI layer can cover the HUD strip (e.g. a full-screen pause menu), but the HUD strip is otherwise a guaranteed "non-playable" zone that actors and background blits never touch.

Configuration

`hud_border` is nested inside the scene's `camera` block:

```
"camera": {  
  "mode": "follow",  
  "target": "player",  
  "hud_border": {  
    "top": 16,  
    "bottom": 0,  
    "left": 0,  
    "right": 0,  
    "bg_color_index": 3  
  }  
}
```

Field	Type	Default	Description
<code>top</code>	int	<code>0</code>	Pixels reserved at the top of the framebuffer for HUD.
<code>bottom</code>	int	<code>0</code>	Pixels reserved at the bottom.
<code>left</code>	int	<code>0</code>	Pixels reserved on the left.
<code>right</code>	int	<code>0</code>	Pixels reserved on the right.
<code>bg_color_index</code>	int	<code>-1</code>	Palette index to fill the HUD strip once at scene start, before <code>_hud_init</code> . <code>-1</code> = don't fill (HUD starts with whatever <code>cls()</code> left). <code>31</code> collapses to <code>-1</code> .
<code>overlay</code>	bool	<code>false</code>	See Overlay mode below.

Valid ranges:

- `top`, `bottom` $\in [0, 61]$ (i.e. `kSceneH/2 - 1`)
- `left`, `right` $\in [0, 81]$ (i.e. `kSceneW/2 - 1`)
- `top + bottom  $\leq$  kSceneH - 8` and `left + right  $\leq$  kSceneW - 8` (at least 8 px of usable camera)

The firmware clamps values to these ranges; TurtleStudio validates on save.

Per-scene: each scene has its own `hud_border`. A menu scene can have no HUD, a gameplay scene can have a top strip, a boss fight can have a wide border.

Absent or `null` fields default to `0` – no change in behavior for existing exported carts.

Playfield

With `T=top, B=bottom, L=left, R=right`, the effective camera playfield is:

```
playfield_w = kSceneW - L - R    (kSceneW = 164)
playfield_h = kSceneH - T - B    (kSceneH = 124)
```

Playfield top-left in framebuffer: (L, T)

The world is sized against the playfield, not the full viewport:

```
world_w = playfield_w * world_steps_x
world_h = playfield_h * world_steps_y
```

Example: `hud_border.top = 16`, `world_steps_y = 1` → world is `164 × 108`.

The scene floor (`y = 0`) anchors to the bottom edge of the playfield.

The HUD region is the complement of the playfield: any pixel with `x < L`, `x ≥ kSceneW - R`, `y < T`, or `y ≥ kSceneH - B`. It can be a top strip, an L-shape, a U-shape, or a full frame depending on which edges are non-zero.

NOTE

`W` and `H` in Lua remain `164` and `124` (full framebuffer dimensions). They are not reduced per scene.

Tile grid

The tile grid is always authored against the canonical viewport (`kSceneW × kSceneH × steps`), regardless of `hud_border`. Tile cells whose scene-Y range falls outside the effective world are clipped pixel-by-pixel at draw time; their visible portion still draws normally. TurtleStudio and firmware agree on the row/column count.

Camera, collision, and Lua coordinates

Follow/clamp, `margin_x/y`, `move()`, `posx()` / `posy()`, and sprite clip all operate against the effective world (`playfield × steps`). Actors cannot be positioned outside the effective world – they are clamped. Scene coordinates are unchanged: `(0, 0)` = bottom-left of the effective world, Y upward.

`spix(sx, sy, c)` in the ENTRY VM works in scene coordinates against the playfield (`sx` in `[0, playfield_w]`, `sy` in `[0, playfield_h]`). Out-of-playfield writes are no-ops.

Overlay mode

When `overlay: true`:

- The world keeps its canonical full size (`kSceneW × world_steps_x`, `kSceneH × world_steps_y`) instead of shrinking to the playfield.
- The camera clamps against the full framebuffer rather than the playfield, so it does not auto-scroll to reveal an actor who walks into the HUD strip.
- An actor positioned in rows above `playfield_h` (e.g. jumping above the top HUD strip) has its sprite invisibly clipped by the playfield clip on scene blits – the actor disappears behind the HUD strip without the view scrolling, Metroid-style.
- The HUD, painted after actors, is unaffected.

`overlay: false` (the default) preserves the prior behavior: the world equals the playfield, and actors bounce against the inner playfield boundary.

Lua API — ENTRY VM

The ENTRY VM stays alive for the entire cart execution (it is not destroyed after the boot script finishes). It owns the HUD; actor

scripts have no access to `hud_*` bindings.

Drawing functions

All coordinates are framebuffer-space – `(0, 0)` = top-left, Y downward. Any pixel that falls inside the current playfield is a no-op – this prevents accidentally painting over the game area.

Function	Description
<code>hud_pix(x, y, color_index)</code>	Draw one pixel in the HUD region.
<code>hud_rect(x, y, w, h, color_index)</code>	Fill a solid rectangle in the HUD region.
<code>hud_clear([color_index])</code>	Fill the entire HUD region with <code>color_index</code> (default <code>0</code>).
<code>hud_text(x, y, str, font_id [, color_index])</code>	Draw <code>str</code> using <code>font_id</code> . Same color-tint semantics as <code>text</code> in ENTRY. Returns width drawn in pixels.
<code>hud_text_width(str, font_id)</code>	Measure text width without drawing.

Fonts resolve from the bundle's `.tfn` assets – the same font file works for both HUD and game text.

There is no `hud_sprite` in v0. Sprite-heavy HUDs (animated icons, portrait frames) belong in [GUI layers](#).

Lifecycle hooks

Both hooks are optional. If undefined, the cart pays zero HUD cost for them.

```
function _hud_init()
```

Called once when the scene starts, after static background/tiles/text labels are drawn and **before** the static snapshot is taken. Output becomes part of the static snapshot – it survives the actor dirty-rect mechanism with no per-frame repaint cost on fixed-camera scenes.

```
function _hud(dt)
```

Called once per frame, **after** `draw_all_actors` and **before** `flip()`. `dt` is seconds (same as actors). Use for animated HUDs: blinking text, live counters, health bars.

WARNING

Do not call `cls()`, `flip()`, or any scene-drawing function inside `_hud`. Only `hud_*` functions are safe here.

Inside both hooks, all standard ENTRY globals are available: `btn`, `btnp`, `text`, `text_width`, `print`, `W`, `H`, `COLORS`, `gui_layer_*`, etc.

Interaction with the static snapshot

Fixed camera (no scroll): the firmware uses `snapshot_static` + `restore_static_dirty`. `_hud_init` runs before the snapshot, so the HUD is baked into the static layer. Subsequent `hud_*` calls (in `_hud`) update both the live framebuffer and the static layer (`s_static_fb`) and mark those cells dirty – so `restore_static_dirty` never reverts the HUD state on the next frame.

Scrolling camera: the firmware repaints the scene every frame (`paint_scene_static_layers` + full flip). `_hud_init` runs after the first repaint. The HUD persists as long as Lua doesn't overwrite it – background/tile and actor blits never touch the HUD strip. Carts typically define `_hud(dt)` to animate and leave `_hud_init` for initial decoration.

Per-frame compositing order

1. `paint_scene_static_layers` – background + tiles + scene text labels → playfield only
 2. `draw_all_actors` – sprites → playfield only
 3. `_hud(dt)` – HUD strip updates
 4. GUI layers (method 2), sorted by `z`
 5. `flip()` – display
-

TurtleStudio authoring

- Scene editor exposes `top`, `bottom`, `left`, `right` as numeric inputs in the same camera group as `mode` and `margin_x/y`.
 - Canvas preview draws a **semi-transparent overlay** over the HUD strips so authors see exactly what the playfield looks like.
 - The Play simulator (`play_runtime.py`) applies the same clip and offset as the firmware – behavior in the editor matches hardware.
 - Migration: existing scenes without `hud_border` behave as before (`{0,0,0,0}`). The field is omitted from the JSON when all values are zero to keep diffs clean.
-

Common problems

Symptom	Likely cause
HUD visible for one frame then gone	Scene uses <code>follow</code> camera with scroll – no static snapshot. Repaint the HUD in <code>_hud</code> each frame, or call <code>_hud_init</code> logic from <code>_hud</code> once with a guard flag.
HUD flickers between old and new values	Only the changed part is updated but not cleared first – call <code>hud_rect</code> to erase the region before repainting it.

Symptom	Likely cause
Player sprite clipped at the top of the playfield	Confusion between framebuffer-Y (down) and scene-Y (up). <code>hud_border.top</code> reserves rows at the framebuffer top , which lowers the maximum scene-Y actors can reach. The actor is still fully visible inside the playfield.
Previous scene's HUD persists after scene change	This shouldn't happen – <code>turtle_gpu_cls</code> in <code>turtle_scene_begin_runtime</code> clears the entire framebuffer including the HUD strip. Repaint in <code>_hud_init</code> of the new scene if you want the same HUD to continue.

Out of scope in v0

- `hud_sprite(x, y, sprite_id, frame)` – sprite blits in the HUD strip (planned for v1; use `hud_rect` + symbol fonts in the meantime)
- Alpha / blending in the HUD region (index 31 = transparent, same as everywhere else)
- Multiple simultaneous palettes (HUD shares the scene's single active palette)
- GUI layer method 2 interaction beyond covering the HUD strip – see [GUI Layers](#)



GUI



GUI Layers

GUI Layers (v0)

GUI layers are stackable overlays defined in the cart bundle and shown/hidden at runtime from Lua. They handle menus, dialogues, inventory screens, title cards, game-over screens, and non-modal popups – any transient UI that doesn't fit in the permanent HUD strip.



RELATION TO THE HUD BORDER

The HUD border strip is always visible during gameplay. GUI layers are transient and hidden when the game runs alone. A GUI layer **can** cover the HUD strip (e.g. a full-screen pause menu covers everything) – each layer decides its own rectangle. The two systems are independent.

Capabilities in v0

Feature	Support
Content	Solid rectangles, text labels (<code>.tfn</code> font, optional palette tint), progress bars, pip bars, sprite icons
Tiles	Not in v0
Transparency	<code>transparent_bg</code> skips the background fill; index 31 is transparent in glyphs and sprites
Animation	Not built-in – update text/values from Lua each frame
Max visible simultaneously	8 layers

Feature	Support
Z-order	<code>z</code> field (higher = drawn on top); ties resolved by manifest order
Persistence	Reset to hidden at every scene start; restored via <code>gui_layers_autoshow</code> or Lua calls

Bundle format

GUI layers live outside the `scenes` block – they form a global catalogue that any scene can reference by id.

```
{
  "guilayers": [
    {
      "id": "pause_menu",
      "x": 0, "y": 0, "w": 164, "h": 124,
      "bg_color_index": 0,
      "transparent_bg": false,
      "pauses_scene": true,
      "captures_input": true,
      "z": 100,
      "rects": [...],
      "text_labels": [...],
      "progress_bars": [...],
      "pip_bars": [...],
      "sprites": [...]
    }
  ]
}
```

Layer fields

Field	Type	Default	Description
<code>id</code>	string	required	Unique id in the catalogue. Start with a letter, then letters/digits/ <code>_</code> / <code>-</code> , max 32

Field	Type	Default	Description
			chars.
<code>x</code> , <code>y</code>	int	<code>0</code> , <code>0</code>	Top-left corner in framebuffer coordinates (Y down, <code>(0,0)</code> = top-left).
<code>w</code> , <code>h</code>	int	<code>164</code> , <code>124</code>	Size in pixels. Clamped to <code>[1, kSceneW]</code> / <code>[1, kSceneH]</code> .
<code>bg_color_index</code>	int	<code>0</code>	Palette index for the background fill. Ignored when <code>transparent_bg=true</code> .
<code>transparent_bg</code>	bool	<code>false</code>	Skip the background fill – the scene beneath shows through.
<code>pauses_scene</code>	bool	<code>false</code>	While any visible layer has this set, actor <code>_update(dt)</code> and <code>move()</code> are suspended. See Pause .
<code>captures_input</code>	bool	<code>false</code>	While any visible layer has this set, <code>btn</code> / <code>btnp</code> return <code>false</code> in actor VMs. See Input capture .
<code>z</code>	int	<code>0</code>	Paint order. Higher = drawn on top.
<code>rects</code>	array	<code>[]</code>	Solid filled rectangles. Max 16 per layer.
<code>text_labels</code>	array	<code>[]</code>	Text labels with font and optional color tint. Max 16 per layer.
<code>progress_bars</code>	array	<code>[]</code>	Fractional fill bars. Max 4 per layer.
<code>pip_bars</code>	array	<code>[]</code>	Discrete icon bars. Max 4 per layer.
<code>sprites</code>	array	<code>[]</code>	Static sprite icons. Max 4 per layer.

Content elements

Rectangles (`rects`)

Solid filled rectangles inside the layer. Painted in array order (index 0 first, last index on top).

Field	Type	Default	Description
<code>x</code> , <code>y</code>	int	<code>0</code>	Relative to the layer's <code>(x, y)</code> . Clamped to layer bounds.
<code>w</code> , <code>h</code>	int	<code>1</code>	Size in pixels. Clamped to remaining space from <code>(x, y)</code> .
<code>color_index</code>	int	<code>0</code>	Palette index. Index 31 is a no-op (transparent).

Text labels (`text_labels`)

Field	Type	Default	Description
<code>id</code>	string	required	Identifier within this layer, used by <code>gui_layer_set_text</code> . Max 32 chars.
<code>x</code> , <code>y</code>	int	<code>0</code> , <code>0</code>	Relative to the layer's <code>(x, y)</code> . Top-left of the first glyph (Y down).
<code>font</code>	string	required	Stem of a <code>.tfn</code> font in the bundle. If missing, the label is skipped with a Serial warning.
<code>text</code>	string	<code>" "</code>	Initial content. Runtime buffer: 63 chars max (64 bytes including nul).
<code>color_index</code>	int	<code>-1</code>	<code>-1</code> = no tint (glyph's own palette colors). <code>0..30</code> = flat tint applied to every non-transparent glyph pixel.

Progress bars (`progress_bars`)

Fractional fill in one direction. Value is expressed as `value_num / value_den` (integers, no floats in firmware). The filled area is `round(fill_dim * fraction)` where `fill_dim` is `w` for horizontal directions or `h` for vertical.

Drawn **after** rects and **before** text labels (so text can overlay the bar as a visible value label).

Field	Type	Default	Description
<code>id</code>	string	required	Unique within this layer. Used by <code>gui_layer_set_progress</code> .
<code>x</code> , <code>y</code>	int	<code>0</code> , <code>0</code>	Relative to the layer's <code>(x, y)</code> .
<code>w</code> , <code>h</code>	int	<code>1</code> , <code>1</code>	Bar rectangle size in pixels.
<code>direction</code>	string	<code>left_to_right</code>	One of: <code>left_to_right</code> , <code>right_to_left</code> , <code>top_to_bottom</code> , <code>bottom_to_top</code> .
<code>fill_mode</code>	string	<code>color</code>	<code>color</code> (solid fill) or <code>sprite</code> (tiled sprite fill).
<code>fill_color_index</code>	int	<code>11</code>	Palette index for the fill when <code>fill_mode="color"</code> . Index 31 = no-op.
<code>fill_sprite_id</code>	string	<code>""</code>	Sprite stem when <code>fill_mode="sprite"</code> . Tiled across the filled area; partial tiles are clipped. Index 31 pixels are transparent.

Field	Type	Default	Description
<code>bg_color_index</code>	int	<code>3</code>	Palette index for the empty part of the bar. Use <code>31</code> to let the scene beneath show through.
<code>border_color_index</code>	int	<code>-1</code>	<code>-1</code> = no border. <code>0..30</code> = 1 px outline around the full bar rectangle.
<code>value_num</code>	int	<code>0</code>	Numerator of the current value. Range <code>[-32768, 32767]</code> .
<code>value_den</code>	int	<code>1</code>	Denominator (maximum). Range <code>[1, 32767]</code> . Zero or negative collapses to <code>1</code> .
<code>ranges</code>	array	<code>[]</code>	Value-range color/sprite overrides. Max 3 per bar. See Value ranges .

Pip bars (`pipBars`)

N discrete icons showing an integer value – HP hearts, keys, medals. Each "on" pip (index `< value`) blits `sprite_full_id`. "Off" pips are **not drawn** (the background shows through), so you can place a static sprite or rect in `rects` to render the empty-pip look.

Field	Type	Default	Description
<code>id</code>	string	required	Unique within this layer. Used by <code>gui_layer_set_pips</code> .
<code>x</code> , <code>y</code>	int	<code>0</code> , <code>0</code>	Relative to the layer's <code>(x, y)</code> . Top-left of the first pip.

Field	Type	Default	Description
<code>sprite_full_id</code>	string	required	Sprite stem for the "on" state. Its pixel dimensions set the pip width/height. Index 31 = transparent.
<code>direction</code>	string	<code>horizontal</code>	<code>horizontal</code> (pips grow toward +x) or <code>vertical</code> (toward +y).
<code>gap_px</code>	int	<code>0</code>	Spacing between consecutive pips. Range <code>[0, 32]</code> .
<code>value</code>	int	<code>0</code>	Number of "on" pips to draw. Clamped to <code>[0, max_value]</code> .
<code>max_value</code>	int	<code>1</code>	Total pip count. Range <code>[1, 32]</code> .
<code>ranges</code>	array	<code>[]</code>	Value-range sprite overrides. Max 3 per bar. See Value ranges .

Sprite icons (`sprites`)

A 1:1 blit of a single sprite at a fixed position. Intended for HUD iconography – a gear icon next to a counter, a player face next to the lives display, a status icon next to a timer. The layer does not animate the sprite; Lua can rotate frames via `gui_layer_set_sprite`.

Drawn **after** pip bars and **before** text labels.

Field	Type	Default	Description
<code>id</code>	string	required	Unique within this layer. Used by <code>gui_layer_set_sprite</code> .
<code>x</code> , <code>y</code>	int	<code>0</code> , <code>0</code>	Relative to the layer's <code>(x, y)</code> . Top-left of the blit.

Field	Type	Default	Description
<code>sprite_id</code>	string	required	Sprite stem in the bundle.
<code>frame_index</code>	int	<code>0</code>	Which frame of a multi-frame sprite to draw.
<code>flip_h</code>	bool	<code>false</code>	Horizontal mirror at blit time.
<code>flip_v</code>	bool	<code>false</code>	Vertical mirror at blit time.

Value ranges (`ranges`)

Both progress bars and pip bars accept a `ranges` array (max 3 entries) that **overrides** the base color or sprite when the current value fraction falls inside `[min_pct, max_pct)`. Classic pattern: green above 50%, yellow 25-50%, red below 25%.

```
"ranges": [
  { "min_pct": 0, "max_pct": 25, "alt_color_index": 8 },
  { "min_pct": 25, "max_pct": 50, "alt_color_index": 9 }
]
```

Field	Type	Default	Description
<code>min_pct</code>	int	<code>0</code>	Inclusive lower bound. Range <code>[0, 100]</code> .
<code>max_pct</code>	int	<code>100</code>	Exclusive upper bound (inclusive when <code>max_pct=100</code> , so 100% is never orphaned).
<code>alt_color_index</code>	int	<code>-1</code>	Progress bars with <code>fill_mode="color"</code> only. <code>-1</code> = don't override.
<code>alt_sprite_id</code>	string	<code>""</code>	Progress with <code>fill_mode="sprite"</code> : replaces <code>fill_sprite_id</code> . Pip bar: replaces <code>sprite_full_id</code> . <code>""</code> = don't override.

Ranges are evaluated in array order – the **first** match wins. If no range matches, the bar's base color/sprite is used. Ranges with `min_pct >= max_pct` are discarded at parse time.

Lua API

All functions operate on the catalogue loaded from the current bundle. An unknown `id` or sub-id is a **silent no-op** (the cart can call without existence checks). The first failure per id is logged to Serial for debugging.

Function	Description
<code>gui_layer_show(id [, z])</code>	Mark the layer visible. Optional <code>z</code> overrides the manifest <code>z</code> for this show only.
<code>gui_layer_hide(id)</code>	Mark the layer hidden. No-op if already hidden.
<code>gui_layer_visible(id) → bool</code>	Query visibility.
<code>gui_layer_set_text(id, label_id, str)</code>	Update a text label. Truncated to 63 chars. Persists until next call.
<code>gui_layer_set_progress(id, bar_id, num [, den])</code>	Update <code>value_num</code> of a progress bar. Optional <code>den</code> also replaces <code>value_den</code> (useful when the maximum changes at runtime, e.g. a level-up raises max HP).
<code>gui_layer_set_pips(id, bar_id, val [, max])</code>	Update <code>value</code> of a pip bar. Optional <code>max</code> replaces <code>max_value</code> . <code>val</code> is clamped to <code>[0, max_value]</code> after the update.
<code>gui_layer_set_sprite(id, icon_id, sprite_id [, frame])</code>	Replace the <code>sprite_id</code> (and optionally <code>frame_index</code>) of a sprite icon. Useful for dynamic iconography (key present/absent, player face by state).

Function	Description
<code>gui_layer_hide_all()</code>	Hide all currently visible layers. Useful for transitions and state changes.

Auto-show per scene

Each scene can declare a `gui_layers_autoshow` array of layer ids to show automatically when the scene starts – without any Lua code. Intended for persistent level HUDs (score, lives, radar). Modal layers like pause menus or dialogues should still be Lua-triggered and should **not** appear here.

```
{
  "id": "level_1",
  "gui_layers_autoshow": ["hud_score", "hud_lives"],
  ...
}
```

- Applied **after** the scene visibility reset and **before** the first `_hud(dt)` tick.
- Equivalent to calling `gui_layer_show(id)` without a `z` override.
- The cart can call `gui_layer_hide(id)` at any time to hide an auto-shown layer (e.g. hide the HUD during a cutscene).
- Unknown ids: no-op with a Serial warning. Duplicate ids: silently ignored.
- Scenes without this field default to `[]` (no auto-shown layers).

In TurtleStudio, the scene editor shows a checkbox per layer in the `guilayers` catalogue; checking it adds the id to `gui_layers_autoshow`.

Pause

When any visible layer has `pauses_scene: true`, on the next tick:

- Actor `_update(dt)` calls are skipped.
 - Actor sprite animation continues (frames still advance). To fully freeze a sprite, call `play_anim(..., speed=0)` before showing the layer.
 - `move()` does not run (nothing calls it without `_update`).
 - Scene text label blinking continues.
 - `_hud(dt)` in the ENTRY VM continues – so the cart can navigate the menu and update the HUD.
-

Input capture

When any visible layer has `captures_input: true`:

- `btn(k)` / `btnp(k)` return `false` for all buttons in actor VMs – actors see no input.
- The ENTRY VM still receives normal `btn` / `btnp` – `_hud(dt)` can navigate the menu.

Set `captures_input: false` (the default) for non-blocking popups like "Achievement unlocked!" where the player should keep moving.

Compositing order (per frame)

1. `paint_scene_static_layers` – world background + tiles + scene text labels → playfield
2. `draw_all_actors` – sprites inside the playfield
3. `_hud(dt)` – HUD strip (method 1)
4. GUI layers, sorted by `z` ascending
5. `flip()` – send framebuffer to display

GUI layers use `turtle_gpu_pixel_raw` internally, which bypasses the playfield write guard that protects the game area from HUD bindings. This is what allows a layer to cover the playfield with a pause menu.

Common problems

Symptom	Likely cause
<code>gui_layer_show("pause")</code> shows nothing	The id doesn't match – typo, or the layer isn't in the bundle. Check Serial output at boot.
Layer appears for one frame then vanishes	Something else is calling <code>gui_layer_hide</code> or <code>hide_all</code> in the same tick.
HUD strip invisible under a layer	Expected – the layer covers it. Set <code>transparent_bg: true</code> to let the HUD show through.
Actor keeps moving with the pause menu up	The layer doesn't have <code>pauses_scene: true</code> . Update the manifest and re-export.
Popup freezes the player	The layer has <code>pauses_scene</code> or <code>captures_input</code> set to <code>true</code> . Set both to <code>false</code> .
<code>gui_layer_set_text</code> has no effect	<code>label_id</code> doesn't match any label in that layer. Check for typos.

Out of scope in v0

- Direct Lua sprite blits (`gui_layer_blit_sprite`) – sprite icons are declarative; the cart can swap sprite/frame via `gui_layer_set_sprite`
- Tile layers inside GUI layers (planned for v2)
- Built-in text blink or sprite animation – simulate from `_hud(dt)` with `gui_layer_set_text` / `gui_layer_set_sprite`
- Layer fade/transition effects

- Dynamic layout (center, padding) – all positions are fixed in the manifest
- Full-stop pause including sprite animation – call `play_anim(..., speed=0)` before showing the layer

TurtleStudio

TurtleStudio is the Python authoring tool for TurtleReader projects. It ships a CLI and a Dear PyGui GUI editor.

Installation

```
cd tools/turtlestudio
python3 -m venv .venv && source .venv/bin/activate
pip install -e .
```

Launching the GUI

```
python -m turtlestudio gui
# or, after editable install:
turtlestudio gui
```

CLI reference

```
# Show help
PYTHONPATH=src python3 -m turtlestudio --help

# Build a project into a .turtlecart + SD package
PYTHONPATH=src python3 -m turtlestudio build path/to/main.lua \
    -o out.turtlecart [--palette pal.txt] [--entry name.lua]

# Initialize a new project directory
PYTHONPATH=src python3 -m turtlestudio project init path/to/project \
    [--name MyGame] [--force]
```

Project layout on disk

A TurtleStudio project is a directory with:

```

turtlestudio.json      # master manifest (source of truth when open)
palettes/              # .pal files (palette definitions)
scenes/<id>.json        # scene data (mirrors of manifest, for review)
objects/
  Objects/<id>.json     # object definitions (animations, collision,
script)                #
  Sprites/<id>.json     # sprite frame data (JSON, pre-export)
tiles/<id>.json         # tileset definitions
scripts/
  global.lua           # ENTRY script (embedded in main.turtlecart)
  <stem>.lua           # actor/scene scripts

```

NOTE

`turtlestudio.json` is authoritative while the project is open.
`scenes/*.json` are mirrors for external review – do not edit them directly.

Export (build)

Exporting bakes the project into the microSD-ready `build/` folder:

1. Scans all scene assets
2. Renders sprites, tilesets, and backgrounds to binary (`asset_bin.py` via numpy)
3. Writes `main.turtlecart` (embeds ENTRY Lua, references sidecar bundle)
4. Writes `studio/project_bundle.json` manifest
5. Copies `scripts/*.lua` and `objects/*.json`

Copy the entire `build/` folder to the root of the microSD card.

Verification scripts

```

# Unit tests – tile collision parsing
PYTHONPATH=src python3 src/turtlestudio/test_tile_collision.py

# Build + verify a project (defaults to exampleprojects/demo1)

```

```
PYTHONPATH=src python3 src/turtlestudio/test_asset_bin.py [project_dir]

# Sanity-check an exported SD package
PYTHONPATH=src python3 src/turtlestudio/verify_package.py path/to/build
```

Key modules

Module	Purpose
<code>project.py</code> / <code>project_runtime.py</code>	Manifest loading + in-memory project state
<code>build.py</code>	Writes <code>main.turtlecart</code>
<code>asset_bin.py</code> / <code>asset_bin_decode.py</code>	Binary asset encoder/decoder
<code>sprites.py</code> / <code>backgrounds.py</code> / <code>tiles.py</code>	Per-asset-type JSON serialization
<code>tile_collision.py</code>	Tile collision shape serialization
<code>scene_camera.py</code> / <code>scene_tiles.py</code>	Scene editing logic
<code>gui.py</code>	Dear PyGui window and editor panels
<code>cli.py</code>	argparse CLI entry point
<code>fonts.py</code>	Font handling (<code>LATIN_CHARSET</code> definition)

Asset naming rules

IDs and script stems share one naming rule across the tool and firmware:

- Starts with a **letter**
- Then letters, digits, `_`, or `-`

- Max 64 characters

Starter Lua template

When you initialize a project or create a script in the Objects tab, TurtleStudio generates a starter `global.lua`:

```
-- scripts/global.lua - runs once at boot (ENTRY VM)
print("TurtleReader: cart loaded")
cls(0)
flip()
```

Object script template

From the Objects tab → Create .lua in scripts/:

```
-- scripts/<stem>.lua - runs every frame (Actor VM)
function _update(dt)
    -- dt: seconds since previous frame
end
```

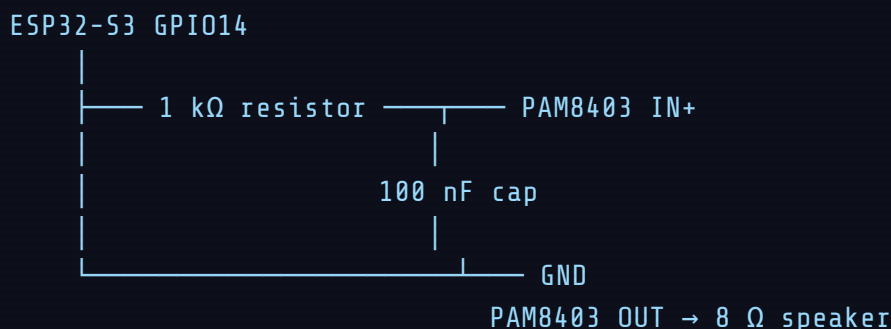
Architecture notes

- TurtleStudio (PyQt6 + Dear PyGui) is fully independent from the pygame runtime
- Both share only the `tortoisengine` data layer (*Note: TurtleStudio is for the ESP32 cart; TortoiseMecha/`tortoisengine` is the separate SBC console – do not mix*)
- `watchdog` is a listed dependency for future hot-reload support but is not yet wired up

Audio — M0 Hardware Bringup

This page covers the M0 (initial) hardware bring-up for audio on the ESP32-S3 target. It documents the signal chain, component selection, and verification sketch. Full Lua API integration is out of scope for M0.

Signal chain



Component	Value / Part	Notes
RC filter resistor	1 kΩ	Between GPIO and cap
RC filter cap	100 nF	To GND; sets cutoff ~1.6 kHz
Amplifier	PAM8403	Filterless class-D, 3 W per channel @ 4 Ω
Speaker	8 Ω	Typical small panel speaker
Mute pin	GPIO42 (optional)	Active-low; tie HIGH to enable always

The RC low-pass filter ($f_c \approx 1/(2\pi \cdot R \cdot C) = 1.6 \text{ kHz}$) attenuates PWM carrier harmonics before they reach the amplifier. Game audio typically

sits below 4 kHz so this cutoff is acceptable for M0 bringup; a wider filter can be used once the full audio pipeline is defined.

PWM generation

The firmware uses the ESP32-S3 LEDC peripheral (LED Control, repurposed as general PWM) to generate the audio signal on GPIO14.

Parameter	Value
GPIO	14
LEDC channel	0
Timer	LEDC_TIMER_0
Resolution	8-bit (256 duty levels)
Carrier frequency	Configurable; audio-range sample rate
Mute GPIO	42 (active-low, optional)

Bringup sketch

`firmware/TurtleReader/AudioBringup.ino` plays a sequence to verify the chain without needing a full cartridge:

1. 440 Hz tone – concert A, ~1 second
2. 880 Hz tone – one octave up, ~1 second
3. C major scale – C4, D4, E4, F4, G4, A4, B4, C5, ~300 ms each
4. 100 Hz → 4 kHz sweep – linear frequency sweep over ~3 seconds

If all four steps produce audible sound without distortion the signal chain is verified.

Running the sketch

```
# Open in Arduino IDE – same board settings as TurtleReader.ino
# Board: ESP32S3 Dev Module
# PSRAM: OPI PSRAM → Enabled
# Serial: 115200
```

Flash and open the serial monitor. Expected output:

```
AudioBringup: starting
Playing 440 Hz...
Playing 880 Hz...
Playing C major scale...
Sweeping 100 Hz → 4000 Hz...
AudioBringup: done
```

Troubleshooting

Symptom	Likely cause
No sound, amplifier powered	GPIO14 not generating PWM – check LEDC init
Loud buzz, no tone	RC filter missing or wrong value
Distortion at all volumes	PAM8403 supply voltage too low (needs 2.5–5 V)
Sound only on one channel	Single-ended wiring – PAM8403 expects differential input for stereo; wire mono to IN+ with IN- to ground
Mute pin has no effect	GPIO42 left floating; pull down or tie to GND explicitly

Out of scope in M0

- Lua `sfx()` / `music()` API bindings
- Wavetable or sample playback
- Stereo output (M0 is mono)
- Volume control from Lua
- Audio mixing with background music

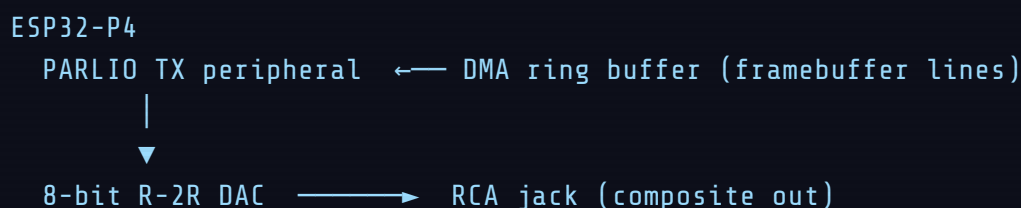
RCA Composite Video Driver (v0)

This page covers the v0 design for the ESP32-P4 composite video output driver. The P4 target outputs NTSC monochrome video over an RCA connector – no LCD, no SPI display. This is a separate firmware sketch (`TurtleReaderP4/`) and a distinct hardware platform from the ESP32-S3 TurtleReader.

NOTE

The composite driver design has not yet been tested against real ESP32-P4 hardware as of v0. All timing and level values are derived from NTSC spec and the PARLIO+GDMA API documentation.

Architecture overview



The **PARLIO TX** peripheral clocks out parallel data at a fixed rate. The output pins are wired to an **R-2R resistor ladder** (8-bit DAC) to produce the analog voltage levels required by composite video. A **GDMA ring** feeds the PARLIO TX with scanline data – each DMA entry contains the samples for one TV line.

Signal parameters

Parameter	Value
Video standard	NTSC (first target)
Output	Monochrome (luminance only)
Clock	5 MHz
Samples per line	320 (64 μ s / 200 ns)
Active video samples	264 (matches canonical scene width)
Lines per frame	262
Frame rate	~59.6 Hz
Sync tip	0 V
Black level	~0.3 V
White level	~1.0 V

Active video spans 264 samples per line – matching the engine's 264-wide framebuffer – leaving the remaining samples for blanking and sync.

PAL support

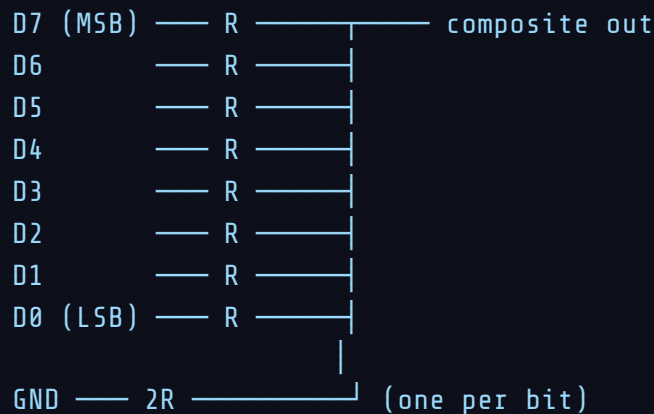
`TURTLE_VIDEO_STD` is an enum in `turtle_gpu_p4.h`:

```
enum TurtleVideoStd { TURTLE_VIDEO_NTSC, TURTLE_VIDEO_PAL };
```

PAL timing (312 lines, 50 Hz, 5.5 MHz clock) is reserved for v1. The driver architecture accommodates it without hardware changes.

R-2R DAC wiring

8 ESP32-P4 PARLIO output pins are wired as an R-2R resistor ladder to produce the composite signal. Each bit position doubles the resistor value toward the MSB:



Exact resistor values depend on the target voltage swing and output impedance required by the TV's 75 Ω input. Refer to [spec/rca-composite-driver-v0.md](#) for the full resistor table.

DMA ring buffer

Each DMA descriptor covers one TV line (320 bytes). The ring contains:

- **Sync lines** (vertical blanking): samples set to sync tip (0x00)
- **Blanking + active lines**: left blanking, active video region (264 bytes from framebuffer row), right blanking
- **Sync pulse** embedded at the start of each line for horizontal sync

The GDMA DMA-chain is set up as a linked list of descriptors that loops continuously. The CPU (or a background task) updates the framebuffer region of each descriptor's buffer for the next frame while the current frame is transmitting.

Bringup stages

Stage 1 – Sync only

Output sync pulses with flat black active area. A TV should lock and show a stable black screen.

```
PARLIO_TX + GDMA → fixed sync pattern → RCA out  
Verify: TV shows black, stable sync lock
```

Stage 2 – Solid gray

Set all active video samples to a mid-gray value (e.g. 0x7F). TV shows stable gray field.

```
Verify: uniform gray, no tearing, correct aspect ratio
```

Stage 3 – Framebuffer output

Wire the framebuffer scanlines into the DMA ring. Each active video line reads from `framebuffer[row]`. This stage maps the 264×198 scene to the TV scanlines.

NOTE

Stage 3 is not in the v0 driver – framebuffer integration is planned for v1.

Bringup sketch

`firmware/TurtleReaderP4/TurtleReaderP4_CompositeBringup.ino` runs Stage 1 and Stage 2 in sequence:

1. Init PARLIO TX at 5 MHz, 8-bit parallel
2. Allocate DMA ring in PSRAM (262 descriptors × 320 bytes)
3. Fill ring with NTSC sync pattern

4. Start DMA → TV should lock
5. After 3 seconds, fill active area with gray
6. Report sync status to Serial

Expected Serial output:

```
P4 composite bringup
PARLIO TX init OK
GDMA ring alloc OK (83840 bytes)
Stage 1: sync only – check TV for lock
Stage 2: gray field
Bringup complete
```

Shared cartridge format

The ESP32-P4 target uses the same `.turtlecart` format and `GFX_TIER` compile-time constant as the ESP32-S3 target:

<code>GFX_TIER</code>	Target	Display
<code>0</code>	ESP32-S3	ILI9488 SPI LCD
<code>1</code>	ESP32-P4	NTSC composite RCA

A cartridge must never *require* a tier to load – tiers only change rendering fidelity and performance ceiling. Game logic and Lua scripts are identical across tiers.

Out of scope in v0

- PAL timing
- Color (chroma sub-carrier) – NTSC color requires a 3.58 MHz burst and QAM encoding; out of scope until monochrome is stable

- Framebuffer DMA integration (Stage 3)
- Audio over RCA (composite audio is a separate connector)
- Interlaced output (480i)